Design and Implementation of a Web Application for Intelligent Meal Planning with Local Cuisine Integration

By

Bexultan Yessirkegen (SE-2329)
Temirlan Ilyassov (SE-2312)



School of Software Engineering
Astana IT University

6B06102 — Software Engineering
Supervisor: Alua Myrzakerimova

June 2026
Astana

Abstract

This thesis presents the design, implementation, and operational preparation of an intelligent meal planning system for Kazakhstani households. The system generates weekly plans for households of one to ten members under hard budget, allergy, religious, time, and repetition constraints while jointly optimising preference, diversity, and multi-macronutrient fit. The core contribution is a three-tier hybrid planner: a deterministic candidate-pool builder enforces domain rules and quality gates; an OR-Tools CP-SAT model performs combinatorial assignment of recipes to (day, meal-slot) pairs; and a LightGBM listwise ranker, trained on real and synthetic feedback, replaces the deterministic scoring function when a trained artifact is available, with a graceful fallback chain otherwise. On the reported evaluation scenario, the planner reduces macro-target gaps from thousands of kilocalories and hundreds of grams of protein down to noise. The data layer is a reproducible ETL pipeline joining a Hugging Face recipe corpus, scraped supplementary sources, a United States Department of Agriculture (USDA) FoodData Central nutrition catalog, and a Kazakhstan-specific price baseline derived from official socially-significant-goods statistics and an Arbuz.kz retail snapshot; embedding-based clustering, MinHash deduplication, and an eight-component quality score yield a published catalog of more than ninety thousand recipes. The system is delivered as five sibling codebases coordinated by a single versioned OpenAPI contract: a Django REST backend with asynchronous Celery orchestration, a React Native mobile client built with the Expo managed workflow and structured around twenty-seven screens, a React + Vite web client for landing and marketing, a Terraform-managed infrastructure layer, and a separate ETL repository. The deployment layer is a Terraform-managed Amazon Web Services (AWS) infrastructure for three environments wired through GitHub OpenID Connect (OIDC), ECS Fargate, RDS Multi-AZ, ElastiCache, CloudFront, WAF, and Secrets Manager, accompanied by eight CI/CD workflows, five operational runbooks, and observability through Sentry, PostHog, and CloudWatch. Five Architecture Decision Records justify the lasting choices: monorepo, Fargate, hybrid CP-SAT plus learning-to-rank, Expo managed workflow, and PostgreSQL. The result is a single decision-support application in which classical constraint programming, modern learning-to-rank, and disciplined production engineering are combined end-to-end and reproducibly from the repository state.

Keywords: meal planning, recommender systems, constraint programming, OR-Tools CP-SAT, learning-to-rank, LightGBM, Django, React Native, Expo, AWS ECS Fargate, Terraform, re-

producible bootstrap, household decision support, local cuisine, Kazakhstan.

Dedication and acknowledgements

We dedicate this work to our families, who supported us through long nights of debugging and writing.

We thank our supervisor, Alua Myrzakerimova, for guiding the project from an initial prototype into a coherent end-to-end system, for her patience with iteration cycles, and for insisting on a level of operational discipline that ultimately defined the character of the work.

We thank the staff of the School of Software Engineering at Astana IT University for the academic environment that made this project possible, and our peers for the discussions, code reviews, and informal testing sessions that improved the system across many sprints.

Finally, we acknowledge the open-source community whose contributions underpin every layer of the project: the maintainers of Django, Django REST Framework, Celery, OR-Tools CP-SAT, LightGBM, React Native, Expo, Terraform, and the many smaller libraries on which the system depends.

Authors' declaration

We declare that the work in this diploma work was carried out in accordance with the requirements of the University's Regulations and that it has not been submitted for any other academic award. Except where indicated by specific references in the text, the work is the candidates' own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in the dissertation are those of the authors.

SIGNED (Bexultan Yessirkegen): ..... DATE: .....

SIGNED (Temirlan Ilyassov): ..... DATE: .....

Definitions

Following terms are used in this work:

Household	A unit of decision making composed of one or more members sharing a budget and a common set of meals. In this work a household is modelled by a Group entity with associated GroupMember, MemberConstraint, and GroupPreference records.
Meal Plan	A directed, time-anchored assignment of recipes to a sequence of day-slot pairs over a planning horizon, accompanied by an aggregated shopping list, derived budget and nutrition analytics, and human-readable explanations.
Constraint Programming	A declarative paradigm for solving combinatorial problems by stating variables, their domains, and the constraints between them. The solver searches the joint feasible region for assignments that satisfy all constraints and optionally optimise an objective.
CP-SAT	A hybrid solver combining Boolean Satisfiability search, lazy clause generation, and linear-relaxation techniques. CP-SAT is a component of Google OR-Tools and is the optimisation core of the present planner.
Learning-to-Rank	A class of supervised learning methods that learn a ranking function from labelled examples. The objective is to order items per query rather than to predict an absolute label per item. The present work uses the LightGBM library with the lambdarank objective.

Candidate Pool	The shortlist of recipes that passes deterministic eligibility checks (category, quality score, ingredient and tag blockers, time and budget strictness) before being handed to the optimiser.
Quality Score	A composite scalar in $[0, 1]$ assigned to each recipe by the ETL layer, reflecting eight components: nutrition coverage, ingredient match ratio, total-time presence, yields presence, image presence, step count, parsed-unit ratio, and price-coverage ratio.
Reproducible Bootstrap	The property of a system that allows another developer to recreate a functionally equivalent state from versioned files and a single declared command, without depending on private database dumps or undocumented machine state.
Recovery Point Objective	The maximum amount of recent data that may be lost in the event of an incident, measured as a time window. In this work, RPO is set to one hour for the production database.
Recovery Time Objective	The maximum acceptable time between an incident and full restoration of service. In this work, RTO is set to four hours for the production database.
OpenID Connect	An identity layer on top of OAuth 2.0 used in this work to allow GitHub Actions to assume short-lived AWS roles without storing long-lived access keys in the repository.

Designations and Abbreviations

Following designations and abbreviations are used in this work:

ADR	Architecture Decision Record
ALB	Application Load Balancer (AWS)
API	Application Programming Interface
ACM	AWS Certificate Manager
APN	Apple Push Notification service
AWS	Amazon Web Services
BibTeX	bibliographic database format used by LaTeX
CI/CD	Continuous Integration / Continuous Delivery
CP-SAT	Constraint Programming with SAT-based search (OR-Tools)
CRUD	Create, Read, Update, Delete
DRF	Django REST Framework
DAU	Daily Active Users
DLQ	Dead-Letter Queue
DR	Disaster Recovery
EAS	Expo Application Services
ECR	Elastic Container Registry (AWS)
ECS	Elastic Container Service (AWS)
ETL	Extract, Transform, Load
FCM	Firebase Cloud Messaging

GDPR	General Data Protection Regulation
HF	Hugging Face (recipe dataset host)
IaC	Infrastructure as Code
ILP	Integer Linear Programming
JSON	JavaScript Object Notation
JSONL	JSON Lines
JWT	JSON Web Token
KZT	Kazakhstani Tenge (currency)
LightGBM	Gradient boosting framework used for the ranker
LLM	Large Language Model
LSH	Locality-Sensitive Hashing
ML	Machine Learning
MVP	Minimum Viable Product
NDCG	Normalised Discounted Cumulative Gain
OIDC	OpenID Connect
ORM	Object-Relational Mapping
OTA	Over-the-Air (update)
PITR	Point-in-Time Recovery (RDS)
PR	Pull Request
RDS	Relational Database Service (AWS)
REST	Representational State Transfer
RPO	Recovery Point Objective
RTO	Recovery Time Objective
SDK	Software Development Kit
SE	Software Engineering
SLA	Service Level Agreement
SLO	Service Level Objective

SNS	Simple Notification Service (AWS)
SQS	Simple Queue Service (AWS)
SQL	Structured Query Language
SSM	Systems Manager (AWS)
TLS	Transport Layer Security
ToC	Table of Contents
ToS	Terms of Service
USDA	United States Department of Agriculture
UUID	Universally Unique Identifier
VPC	Virtual Private Cloud (AWS)
WAF	Web Application Firewall (AWS)

List of Tables

Table Page

1.1 Qualitative landscape comparison across eight axes (HHM = household modelling, CON = constraints, BUD = budget, NUT = nutrition, LOC = local cuisine, OPT = optimisation transparency, OPS = production operations, PLT = multi-platform). 9

2.1 Persisted domain entities by Django app. 13

3.1 Estimated first-year deployment cost (USD). 47

4.1 Recipe corpus state after the production-shaped dump is restored. 49
4.2 Deterministic versus CP-SAT planner on the representative scenario. 51
4.3 Four household scenarios and CP-SAT plan-quality summary. 53
4.4 Engineering readiness of the ML scoring layer. 56
4.5 Production-readiness checklist. 57

List of Figures

Figure Page

2.1 C4 Level 1 — system context. 11
2.2 C4 Level 2 — deployable containers and managed services. 12
2.3 Domain ER overview across users, groups, catalog, planning, feedback, and devices apps. Audit-only and link tables are omitted for visual clarity. 14
2.4 etl-meals pipeline from discovery and external datasets to the diploma_work import bundle. 18
2.5 Mobile plan-viewing surfaces: weekly overview, daily detail, recipe detail, and plan explanation. 27
2.6 Mobile analytics, shopping, and pantry surfaces captured from the Expo client. 28

3.1 C4 Level 3 — planning service components from REST request to persisted plan, shopping list, explanation, and push notification. 34

4.1 Recipe category distribution across the bootstrap sample (n = 5,000). 50
4.2 Recipe preparation-time distribution (n = 5,000). 50
4.3 Candidate-pool funnel from all published recipes to the solver shortlist on the representative scenario (logarithmic x axis). 51
4.4 Before-and-after comparison of macro target gaps (lower is better). 52

Table of Contents

Abstract i

Dedication and acknowledgements iii

Authors' declaration iv

Definitions v

Designations and Abbreviations vii

List of Tables x

List of Figures xi

Introduction xvi
- Relevance of intelligent meal planning in household contexts xvi
- Problem statement xvii
- Research questions xvii
- Aim and objectives xviii
- Object, subject, and significance xix
- Scope and non-goals xx
- Structure of the thesis xx

1 Literature Review and Existing Systems 1
- 1.1 Food recommender systems as a research domain 1
- 1.2 Classical recommendation theory 1
- 1.3 Food and recipe recommendation studies 2
- 1.4 Diet optimisation and combinatorial planning 3

1.5 Learning-to-rank in recommendation contexts . . . 3
1.5.1 Internals of the LightGBM ranker . . . 4
1.6 Constraint programming for combinatorial planning . . . 4
1.7 Production-grade machine learning systems . . . 5
1.8 Usability principles for household-scale planning . . . 6
1.9 Cloud-native architectures for ML-enabled applications . . . 6
1.10 Data quality and reference nutrition databases . . . 7
1.11 Local context: food economics in Kazakhstan . . . 7
1.12 Synthesis: position of this work in the literature . . . 8
1.13 Analysis of existing systems . . . 8

2 System Architecture and Data Engineering 10
2.1 Architectural method . . . 10
2.1.1 Five sibling codebases . . . 10
2.1.2 C4 model at three levels . . . 10
2.2 Domain modelling . . . 12
2.2.1 Application boundaries . . . 12
2.2.2 Entity inventory . . . 12
2.2.3 Methodological principle . . . 14
2.3 Reproducible bootstrap . . . 15
2.3.1 Reproduce-from-scratch commands . . . 16
2.4 Data sources landscape . . . 17
2.5 etl-meals pipeline . . . 17
2.6 URL discovery, acquisition, and extraction . . . 19
2.6.1 Discovery and raw acquisition . . . 19
2.6.2 Schema extraction with fallback . . . 19
2.7 Canonical ingredient normalisation . . . 19
2.8 Embedding-based clustering and MinHash deduplication . . . 20
2.9 Quality scoring . . . 20
2.10 USDA nutrition synchronisation . . . 20
2.11 Multi-source price ingestion . . . 21
2.12 Backend implementation surface . . . 21
2.12.1 Public API surface map . . . 22
2.13 Mobile and web clients . . . 26
2.13.1 Mobile client . . . 26

2.13.2 Onboarding workflow 26
2.13.3 Plan viewing and explanation 26
2.13.4 Analytics, shopping, pantry, and sharing 27
2.13.5 Plan generation, pantry, and sharing 28
2.13.6 Feature-Sliced Design in the mobile client 28
2.13.7 Web client 29
2.14 Asynchronous planning pipeline 30
2.14.1 Celery queue topology and idempotency 30
2.15 API design principles 30
2.16 Testing strategy 31
2.17 Database evolution and zero-downtime migrations 32
2.18 Security and compliance model 32

3 Algorithm Design and Production Deployment 34
3.1 End-to-end planning pipeline 34
3.2 Candidate pool construction 35
3.3 Hard-constraint extraction 35
3.4 CP-SAT model formulation 35
3.4.1 Objective weight tuning 36
3.4.2 Solver behaviour in production 37
3.5 Deterministic fallback 37
3.6 Scoring layer: deterministic versus learned 38
3.6.1 Deterministic scorer 38
3.6.2 LightGBM listwise ranker 38
3.6.3 Training data preparation and label sources 39
3.6.4 Cold-start behaviour and graceful degradation 40
3.7 Dynamic servings, shopping list, and explanations 40
3.7.1 Dynamic servings 40
3.7.2 Shopping-list aggregation 40
3.7.3 Plan explanations 41
3.8 Cloud infrastructure 41
3.8.1 Operational goals 41
3.8.2 Terraform module structure 41
3.8.3 Networking 42
3.8.4 Identity, secrets, and least-privilege IAM 43

3.8.5 Threat model and mitigations 43
3.8.6 Compute and data plane 44
3.9 Continuous integration and continuous delivery 44
3.10 ETL throughput, parallelism, and provenance 44
3.11 Observability 45
3.12 Runbooks and disaster recovery 46
3.13 Mobile delivery, store submission, and cost model 46

4 Evaluation, Discussion, and Decisions 48
4.1 Evaluation philosophy 48
4.2 Recipe corpus quality 48
4.3 Catalog distributions 49
4.4 Candidate pool reduction 50
4.5 CP-SAT versus deterministic planner 51
4.6 Additional household scenarios 52
4.7 Sensitivity analysis 53
4.8 User-experience heuristic walkthrough 54
4.9 Observability instrumentation in evaluation 55
4.10 ETL throughput 55
4.11 ML scoring layer readiness 56
4.12 Production readiness checklist 56
4.13 Architecture Decision Records 57
4.14 Threats to validity 59
4.15 Limitations 60
4.16 Future work 60

Conclusion 62

Bibliography 64

Introduction

Relevance of intelligent meal planning in household contexts

Food choice is an everyday necessity, yet the computational interpretation of meal planning as a structured optimisation problem remains an active research domain. A household deciding what to eat across a planning horizon is not selecting individual recipes by taste alone. The decision is shaped by nutritional adequacy for several members with potentially different goals, a shared monetary budget, total available cooking time, ingredient availability, family preferences and dislikes, religious and medical constraints, and the practical sequence of meals so that ingredients can be purchased once and consumed across several days. In digital food systems, these decisions become even harder, because the software must integrate heterogeneous data, encode constraints faithfully, and surface recommendations that are not only mathematically valid but also culturally plausible. Recent surveys of food recommender systems argue that food recommendation is not a straightforward extension of generic recommender systems but a specialised domain with unique sensitivity to health, context, and decision constraints [1, 2].

The relevance of intelligent meal planning is reinforced by the rising importance of preventive health and by the widespread adoption of digital services for nutrition. Nutrition-aware recommendation has been explored in systems such as Yum-Me, which showed that users respond positively when recommendations combine preference learning with nutrient-based reasoning [3]. Earlier work on automatic meal plan generation emphasised that daily or weekly plans must be evaluated differently from isolated recipe suggestions: plan quality depends on aggregate nutrient and repetition effects rather than on the immediate appeal of any single item [4]. Modern systematic reviews further confirm that specialised domains (diabetes management, religious diets, regional cuisine) all require dedicated treatment within food recommender research [2, 5].

In the Kazakhstani context, recent applied work has begun to address local food economics with machine learning, including the use of supervised models to forecast food prices in Kazakhstan [6]. This confirms that local nutrition and food-economics decision support is an active practical concern; the present work approaches the household-planning side of the same problem.

From a software engineering perspective, intelligent meal planning is a particularly suitable thesis topic because it forces the integration of several engineering concerns at once. The system must perform large-scale data acquisition and cleaning across multiple recipe sources, normalise noisy ingredient strings, integrate canonical price and nutrition data, serialise the resulting domain in a relational schema, expose a clean API to client applications, embed both classical optimisation and modern machine learning in a single inference pipeline, develop user-facing software for at least one mobile platform, and stand up a reproducible deployment environment.

Problem statement

The central problem addressed in this thesis is the absence of a localised, explainable, and operationally reproducible meal planning system that can generate household plans under realistic budget, nutrition, and cultural constraints, with first-class support for both mobile and web clients and an infrastructure plan suitable for actual public deployment. Existing tools suffer from one or more weaknesses: they focus on calorie logging rather than plan generation, ignore local cuisines and local prices, use opaque recommendation logic, target individuals rather than households, or stop at prototype-level operational maturity (no asynchronous orchestration, no infrastructure-as-code, no observability, no auditable data pipeline). These limitations make existing tools weak fits for households where several people share a shopping budget, where local cuisine should be a first-class option, and where the planning system is expected to function as a long-running service rather than as a one-off prototype.

Research questions

The research questions of this work are formulated as follows.

RQ1 How can a heterogeneous corpus of public recipe sources, public price data, and

a canonical nutrition reference be reproducibly transformed into a structured catalog that supports household-level cost and nutrition reasoning?

RQ2 How can hard household constraints (budget, allergies, religious bans, time, repetition) and soft preferences (nutritional fit, diversity, cuisine affinity, learned ranking) be combined into a single planning model that is both feasible to solve at interactive latency and inspectable enough to explain to the user?

RQ3 How should the system be decomposed across backend, mobile, web, and ETL codebases so that the algorithmic core remains isolated from delivery concerns while the public API remains a stable contract for both clients?

RQ4 What infrastructure, observability, and release-engineering choices are required to take the prototype from a laptop-only experiment to a deployment-ready system that satisfies recoverability, security, and store-submission requirements?

RQ5 How can the resulting system be evaluated along multiple axes (corpus quality, solver characteristics, deterministic-versus-learned scoring, end-to-end production readiness) in a way that produces honest, reproducible numbers rather than aspirational claims?

Aim and objectives

The aim of this work is to design, implement, and operationally prepare an intelligent meal planning system for Kazakhstani households with hybrid constraint-programming and learning-to-rank scoring. To meet this aim, the work pursues nine objectives.

1. Collect and normalise a multi-source recipe corpus including a ninety-three-thousand-recipe Hugging Face dataset and supplementary scraped sources, then deduplicate and quality-score the result.
2. Integrate two complementary price baselines: an official Kazakhstani statistical reference for socially significant goods, and an Arbuz.kz retail snapshot for long-tail coverage.
3. Synchronise an ingredient-level nutrition catalog from USDA FoodData Central as a stable, citable scientific baseline.

4. Design a relational domain model that captures households, members, constraints, preferences, the recipe catalog, prices, nutrition, planning artifacts, shopping lists, pantry items, feedback, devices, and notifications with explicit boundaries and migrations.

5. Implement a three-tier hybrid planner combining a deterministic candidate-pool builder, an OR-Tools CP-SAT combinatorial assignment, and a LightGBM listwise scoring layer with a robust fallback chain.

6. Build a React Native mobile client (Expo managed workflow) covering the full household workflow from onboarding to plan analytics with push-driven asynchronous job delivery.

7. Maintain a sibling React + Vite web client for landing and marketing that exercises the same public API contract.

8. Prepare a Terraform-managed AWS infrastructure layer for three environments with GitHub OIDC, ECS Fargate, RDS Multi-AZ, ElastiCache, CloudFront, WAF, and Secrets Manager, accompanied by runbooks for deployment, incident response, disaster recovery, secret rotation, and mobile rollback.

9. Evaluate the resulting system using only measurable artifacts that can be reproduced from the repository state, while documenting the validity boundaries of each measurement.

Object, subject, and significance

The object of the research is an intelligent meal planning system addressing household-level decision support. The subject of the research is the architectural, data-engineering, optimisation, and production-engineering methodology required to implement such a system with local cuisine integration and operational maturity beyond prototype scope. The practical significance lies in the fact that the project does not remain at a mockup level. It includes a functioning backend with asynchronous orchestration, a mobile client targeting App Store and Google Play submission, a sibling web client, a multi-source ETL pipeline, an ingredient-level nutrition catalog, a Kazakhstan-specific price baseline, a reproducible bootstrap procedure, a three-tier hybrid planner, and a complete infrastructure-as-code plan for production deployment.

Scope and non-goals

The scope of the project is explicitly bounded. The work covers planning at the weekly horizon for households of one to ten members, with optional pantry awareness and an optional share-link mechanism. The mobile client is store-submission-ready but not yet published. The infrastructure layer is deployment-ready (Terraform plans and GitHub workflows wired through OIDC) but is not currently running in production, because that would require active Apple Developer Program, Google Play Console, and AWS billing arrangements that lie outside the scope of this thesis. The LightGBM ranker pipeline is implemented end to end (feature extraction, dataset building, training, lazy inference, fallback), but offline benchmarking against accumulated real feedback is deferred until production rollout produces sufficient labelled data; the work therefore reports engineering readiness criteria rather than benchmark deltas extracted from synthetic-only data.

Structure of the thesis

The thesis is organised in four chapters preceded by this Introduction. Chapter 1 reviews the literature and analyses existing systems. Chapter 2 presents the system architecture, the data engineering layer, the ETL pipeline, the API surface, and the mobile user flow. Chapter 3 presents the planning algorithm in detail and the cloud-deployment plan. Chapter 4 reports the evaluation, summarises the architecture decisions, and discusses limitations and future work. A Conclusion and references follow.

Chapter 1

Literature Review and Existing Systems

1.1 Food recommender systems as a research domain

The domain of food recommender systems has matured significantly over the last decade. Two recent systematic reviews [1, 2] characterise food recommendation as a specialised branch of recommender systems research with unique domain requirements. Unlike movie or music recommendation, food recommendation must account for health conditions, meal context, ingredient compatibility, cooking skill, and household routines. These requirements change the structure of both the data and the recommendation objective. A recipe that seems appealing from a preference perspective may still be unusable if it violates nutrient goals or cost constraints.

Research in this domain covers multiple paradigms. Content-based methods analyse recipe features and ingredients [7, 8]. Knowledge-based and ontology-driven approaches represent domain semantics explicitly [9–11]. Constraint-based systems model hard user restrictions directly [12, 13]. Health-aware systems incorporate nutrient-based or disease-specific considerations [5, 14]. This diversity demonstrates that no single recommendation paradigm is universally sufficient for food-related decision support.

1.2 Classical recommendation theory

General recommender system theory provides the background needed to interpret food recommendation approaches. Hybrid recommenders combine multiple information sources and are useful when single-strategy recommendation proves insufficient [15].

Broader recommender system surveys describe the major strengths and limitations of collaborative filtering, content-based filtering, and hybrid strategies [16, 17]. Content-based recommendation remains especially relevant in domains where item semantics are rich and explicit, which is why Pazzani and Billsus [18] and Lops et al. [19] are useful conceptual references for recipe and ingredient-based recommendation.

Traditional recommendation theory has to be adapted in the food domain. Collaborative filtering often depends on large-scale historical feedback and performs poorly when user restrictions are explicit or when household-level decisions matter more than individual taste profiles. Constraint-based and knowledge-based approaches therefore become more attractive. Felfernig and Burke [12] argue that constraint-based recommenders are particularly effective when explicit user requirements must shape the recommendation process. This perspective aligns strongly with meal planning, where allergies, dislikes, budgets, and preparation limits cannot be treated as minor preferences.

1.3 Food and recipe recommendation studies

Early food recommendation research focused on recipe similarity and ingredient reasoning [7, 20]. Teng, Lin, and Adamic [8] explored recipe recommendation through ingredient networks, which highlighted the value of ingredient co-occurrence patterns as a signal for food recommendation. Later studies expanded the domain toward personalised health and demographic specificity. Ng and Jin [21] proposed personalised recipe recommendations for toddlers based on nutrient intake and food preferences. Phanich, Pholkul, and Phimoltares [22] considered diabetic patients. Mao, Yuan, Xu, and Wei [23] explicitly considered regional cuisine flavour. Ribeiro et al. [24] focused on Portuguese older adults.

Health-aware recommendation has also advanced in meal-level systems. Yang et al. [3] demonstrated that a nutrient-based meal recommender can combine nutritional objectives with preference learning. Jung and Chung [14] provided a knowledge-based dietary recommendation system for obesity management, while Ueta, Iwakami, and Ito [25] proposed a goal-oriented recipe recommendation system that explicitly exposed nutritional information.

1.4 Diet optimisation and combinatorial planning

Meal planning is closely related to the classical diet optimisation literature. The diet problem formulated in operations research showed that food selection can be cast as an optimisation task. van Dooren [26] reviews the use of linear programming to optimise diets under nutritional, economic, and environmental constraints and emphasises the importance of acceptability constraints. The methodological lesson is that a mathematically valid diet may still be unacceptable to the user if it ignores habitual food patterns or culinary realism; the present project therefore treats optimisation as necessary but not sufficient.

The move from continuous linear programming to expressive combinatorial methods is significant. Once a system must optimise over discrete slots, recipe repetition, category fit, and multi-objective trade-offs, a constraint-programming approach becomes appealing. The CP-SAT-LP solver described by Perron, Didier, and Gay [27] represents a modern development in this area, combining SAT-based search with linear relaxation ideas. For a meal planner that must assign recipes to several day-slot pairs simultaneously while honouring hard constraints, this solver family is theoretically grounded and practically powerful.

1.5 Learning-to-rank in recommendation contexts

Once a planning system grows beyond hand-crafted scoring, learning-to-rank becomes a natural extension. Gradient-boosted decision tree libraries such as LightGBM [28] provide listwise ranking objectives (notably LambdaRank) that align directly with recommendation-style problems where the relative ordering of items per query (or, here, per household) is more important than absolute regression accuracy. The classical ranking metric suite (NDCG@k, MAP, hit-rate) gives a principled vocabulary for comparing a learned ranker against a deterministic baseline, and the listwise formulation makes it natural to train on per-group recipe shortlists. This work adopts LightGBM as the supporting scoring layer precisely because of this fit, while keeping the deterministic scorer as a robust fallback for cold-start operation.

1.5.1 Internals of the LightGBM ranker

LightGBM [29] differs from classical gradient-boosted decision trees in two algorithmic ways that matter for this work. First, Gradient-based One-Side Sampling (GOSS) retains all instances with large absolute gradients and randomly samples instances with small gradients; this approximates information gain while reducing the cost of split-finding by an order of magnitude on large recipe datasets. Second, Exclusive Feature Bundling (EFB) bundles mutually-exclusive sparse features (such as one-hot encodings) into a single denser feature, which is especially useful for the project's category, meal type, and cuisine one-hots. The leaf-wise (best-first) tree growth additionally tends to produce deeper trees on the most informative branches, which captures the highly non-linear interactions between group profile and recipe metadata observed in the training data.

For learning-to-rank, the project uses the lambdarank objective. The conceptual lineage runs from RankNet (pairwise probabilistic loss) through LambdaRank (gradient weighted by an information-retrieval metric, typically NDCG) to LambdaMART (LambdaRank applied to gradient-boosted trees) [30]. The intuition is that the gradient at each pair of items is scaled by the change in NDCG that swapping them would induce; this implicitly biases the model toward correctly ranking items near the top of the list, which matches how a household scrolls through plan options. The evaluation in Chapter 4 reports NDCG@5 and NDCG@10 because these are the cut-offs the user actually sees in the mobile app.

1.6 Constraint programming for combinatorial planning

Constraint programming (CP) is the theoretically appropriate framework for the assignment subproblem that arises after candidate generation. Once the candidate pool has been narrowed to a few dozen recipes per meal slot, the planner must select one recipe per (day, meal) cell so that uniqueness, recipe-repeat caps, and aggregate budget and time are satisfied while a weighted sum of preference scores is maximised. This problem class — assignment under combinatorial constraints with an objective — is well studied in the CP literature. MiniZinc [31] established a standard high-level constraint modelling language that decouples the model from the underlying solver, and most modern CP solvers internally combine SAT-based search with constraint propagation.

Two implementation strategies are relevant for the present work. The classical integer

linear programming (ILP) formulation models each (slot, recipe) pair as a binary variable, with linear constraints on slot uniqueness, category caps, budget, and total preparation time, and an objective that is a linear function of preference scores. ILP is well supported by commercial solvers but has two drawbacks for this domain: discrete logical constraints such as "do not place the same category in two adjacent slots" or "at most one heavy main per day" are unnatural to express linearly, and the search procedure is purely branch-and-bound. The constraint programming with SAT learning formulation (CP-SAT) instead models the same problem with high-level constraint primitives such as AllDifferent and AtMostK, and uses lazy clause generation to compactly encode propagation reasons as clauses that can be reused across the search [32]. OR-Tools' CP-SAT solver [27, 33] blends this CP search with a linear relaxation, yielding a hybrid that is competitive with ILP on linear-friendly instances and clearly better on combinatorially structured instances.

For the meal planner specifically, CP-SAT was chosen for three reasons. First, the natural model is full of discrete logical constraints (no recipe twice, no category twice in two days, soft time targets, exact slot uniqueness) for which CP primitives are more expressive than linear ones. Second, OR-Tools' CP-SAT is permissively licensed, well documented, and embedded directly into the Python process, so there is no separate solver service to operate. Third, the wall-clock budget for plan generation is small (five seconds in production), and CP-SAT's any-time behaviour — it returns the best solution found so far when interrupted — matches this deadline-driven setting well.

1.7 Production-grade machine learning systems

Production machine learning differs from offline modelling in well-known ways. Sculley et al. [34] document the hidden technical debt accumulated by ML systems: configuration sprawl, data dependencies, glue code, and unstable feature pipelines. The implication for this work is to keep the ML scoring layer narrowly scoped (a single (recipe, group) pair mapped to a scalar), to share feature extraction code between training and inference, and to ensure that any failure path falls through to the deterministic scorer. This design discipline is reflected concretely in the project's ML scoring package, which is structured around a single feature extractor reused by both the training and the inference modules.

1.8 Usability principles for household-scale planning

A meal planner that is operationally correct can still fail if the user cannot follow the plan. Nielsen's ten usability heuristics [35] apply directly to a household planning surface. The most relevant heuristics are visibility of system status (the user must understand whether a plan is being generated, ready, or stale), match between system and the real world (Russian-language recipes for a Kazakh audience, metric units, KZT prices), user control and freedom (the planner must support undo through plan archiving and recipe replacement, never trapping the user into a single output), consistency and standards (the same recipe card layout in the home tab, plan tab, shopping list, and shared link), recognition rather than recall (the user should not have to remember constraints from setup; group preferences, allergies, and budget should be visible whenever they influence a recommendation), and help and documentation (per-meal explanations that justify why a recipe was chosen). The mobile design surveyed in Chapter 2 maps each of these heuristics onto specific screens and components.

A second design strand relevant for household planning is transparency as a trust signal. Recommender systems literature has consistently found that opaque algorithmic outputs erode trust even when the outputs are correct [17]. For a meal planner, transparency takes two concrete forms. First, the per-meal rationale (PlanExplanation) must show which constraints and preferences influenced the choice; this is rendered in the plan detail screen as a list of human-readable bullet points. Second, the post-plan analytics screen exposes the cost, nutrition, and balance achieved by the plan, so that the user can audit the planner's behaviour rather than being asked to take the result on faith. Both forms are reflected in the system architecture and APIs presented in Chapter 2.

1.9 Cloud-native architectures for ML-enabled applications

Once a system needs to ship to public app stores and serve household traffic, cloud-native architectural patterns become relevant. Container-based orchestration on managed clouds has become the default for small teams that cannot afford operational complexity overhead. Cluster scheduling research dating back to Borg [36] establishes the conceptual foundation for the managed-compute services that the project consumes (AWS ECS Fargate). The architectural relevance is not in adopting cluster research directly, but in understanding why managed Fargate over Kubernetes is the correct trade-off for a three-service workload: web, worker, and beat. Continuous architecture practice

[37] additionally argues for small, justified decisions captured as Architecture Decision Records (ADRs) rather than monolithic up-front design documents; the project applies this practice through five ADRs summarised in Chapter 4.

1.10 Data quality and reference nutrition databases

Data quality is a recurring issue in food recommendation and nutrition-support systems. Recipe lines on public websites are semi-structured, culturally inconsistent, and full of implicit assumptions. Ingredient-level semantics must often be reconstructed before cost and nutrient calculations are possible. Ontology-based and semantic approaches [9–11] implicitly recognise this issue by emphasising the need for structured food knowledge rather than raw text alone.

For nutritional reference data, USDA FoodData Central offers a structured public baseline. McKillop et al. [38] describe FoodData Central as an integrated food composition system intended to support both research and practical use. This makes USDA especially attractive for a software engineering project that needs reliable and citable nutrient values. The present project additionally confirms a common limitation noted across nutrition-aware systems: even when the nutrient database is high quality, local ingredients and local dish interpretations still require curation at the application level.

1.11 Local context: food economics in Kazakhstan

A small but growing body of applied work targets the Kazakhstani context. Sapakova et al. [6] apply supervised machine-learning models to forecast food prices in Kazakhstan, demonstrating that locally collected data supports practically useful price-prediction tooling. The same paper underscores that the Kazakhstani food market is sufficiently distinctive that imported prediction baselines do not transfer well. This conclusion is consistent with the present work's choice to combine an official Kazakh statistical price reference with a Kazakh retail snapshot (Arbuz.kz), rather than to import generic price data. While the present work targets household planning rather than price prediction, the same locality-first reasoning applies: both nutrition and price baselines benefit from local curation.

1.12 Synthesis: position of this work in the literature

Two threads frame the contribution of this work. First, methodologically, the work follows the constraint-based and knowledge-based traditions in food recommendation [12–14] while complementing them with a learning-to-rank scoring layer [28] guarded by production-discipline principles [34]. Second, infrastructurally, the work commits to ADR-driven cloud-native delivery [36, 37] so that the engineering effort is not lost in a prototype-only artifact. Both threads are connected by the deliberate choice to make the data path itself reproducible: from public recipe sources and USDA references [38] through canonical normalisation and quality scoring to the planner's candidate pool. This combination is rare in the literature: most academic prototypes stop at the algorithmic contribution, and most production ML projects do not document their optimisation core. The present work aims to occupy the intersection.

1.13 Analysis of existing systems

Existing food-related applications and research systems can be grouped into several categories. General recipe portals provide variety and volume but rarely support household-level decision support, and almost never expose budget or nutrition reasoning beyond per-serving labels. Calorie loggers and trackers emphasise personal nutrition logging rather than plan generation. Commercial meal planners (Mealime, Yummly, Plant Jammer, Whisk) come closer to the problem but typically operate on opaque recommendation logic, lack explicit support for local cuisine, and target individual users with at most rudimentary household notions. Health-aware research systems [5, 14, 22] focus on narrow medical conditions and rarely provide the production engineering layer that an end-user product needs. Ontology-driven research systems [9–11] provide deep semantic reasoning but limited operational evaluation. Regional and population-specific systems [21, 23, 24] demonstrate that contextual narrowing improves usefulness but they remain prototype-level deliveries.

This analysis indicates that no single existing category satisfies the combination of transparency, localisation, budgeting, household modelling, and operational maturity required by the present work. The gap is not the absence of any single ingredient but the absence of a delivery that combines them coherently. The qualitative comparison along eight axes is summarised in Table 1.1.

Table 1.1: Qualitative landscape comparison across eight axes (HHM = household modelling, CON = constraints, BUD = budget, NUT = nutrition, LOC = local cuisine, OPT = optimisation transparency, OPS = production operations, PLT = multi-platform).

System	HHM	CON	BUD	NUT	LOC	OPT	OPS	PLT
General recipe portals	–	–	–	part.	–	–	–	varies
Calorie loggers	–	lim.	–	yes	–	–	varies	varies
Commercial planners	lim.	lim.	lim.	part.	no	opaque	yes	yes
Ontology-based [9, 10]	part.	yes	lim.	yes	lim.	yes	no	no
Health-aware [14, 22]	no	yes	no	yes	no	yes	no	no
Regional-specific [23, 24]	lim.	lim.	no	part.	yes	part.	no	no
This work	yes	yes	yes	yes	yes	yes	yes	yes

Two qualifications must be added. First, "yes" in the OPS column for the present work means deployment-ready (Terraform, GitHub OIDC, CloudWatch dashboards, runbooks) rather than currently deployed; this distinction is preserved throughout. Second, the commercial systems were evaluated by their publicly observable behaviour rather than by source-level inspection, so the assessment reflects observable product surface rather than internal architecture. With these qualifications, the gap identified in Table 1.1 provides the motivation for the system design presented in Chapter 2.

Chapter 2

System Architecture and Data Engineering

2.1 Architectural method

2.1.1 Five sibling codebases

The system is decomposed across five sibling codebases: backend/ (Django REST API plus Celery workers), mobile/ (React Native client using Expo), frontend/ (React + Vite web client for landing and marketing), infrastructure/ (Terraform modules and environment configurations), and a separate sibling repository etl-meals/ for the ETL pipeline. This decomposition is the subject of ADR-0001 in Chapter 4. The first four directories live in a single monorepo so that a pull request can touch backend, mobile, and infrastructure together when the API contract changes, reviewers see the full picture in one place, and CI scopes per-directory path filters so that backend changes do not trigger mobile builds. The ETL pipeline lives in a sibling repository because it depends on heavier machine-learning libraries (sentence-transformers, scrapy, datasets) that do not belong in the production runtime image.

2.1.2 C4 model at three levels

The architecture is documented as a C4 model at three levels. The system context (Level 1) places the meal planner between household users, the operator and administrator, app store reviewers, AWS, and external services (USDA, Spoonacular, Hugging Face, Sentry, PostHog, and Apple/Google push gateways). The container view (Level 2) shows the Django web service (Gunicorn behind ALB), Celery worker and beat (queues: default, planning, push), the RDS Postgres Multi-AZ instance, ElastiCache Redis (cache

plus broker on distinct logical databases), S3 media and static buckets fronted by CloudFront, Secrets Manager, the ETL pipeline, the Expo push service, and the two client containers (mobile and web). The component view (Level 3), shown later in Figure 3.1, zooms into the planning service: the API endpoint dispatches to either an inline synchronous path or a PlanGenerationRun record with a Celery enqueue; the orchestrator constructs the candidate pool, runs scoring (deterministic or ML), invokes the CP-SAT model with a deterministic fallback, builds the shopping list, emits explanations, persists the result, and triggers an Expo push to the requesting user. The Mermaid sources are maintained alongside the running code so that the diagrams cannot drift from the implementation.

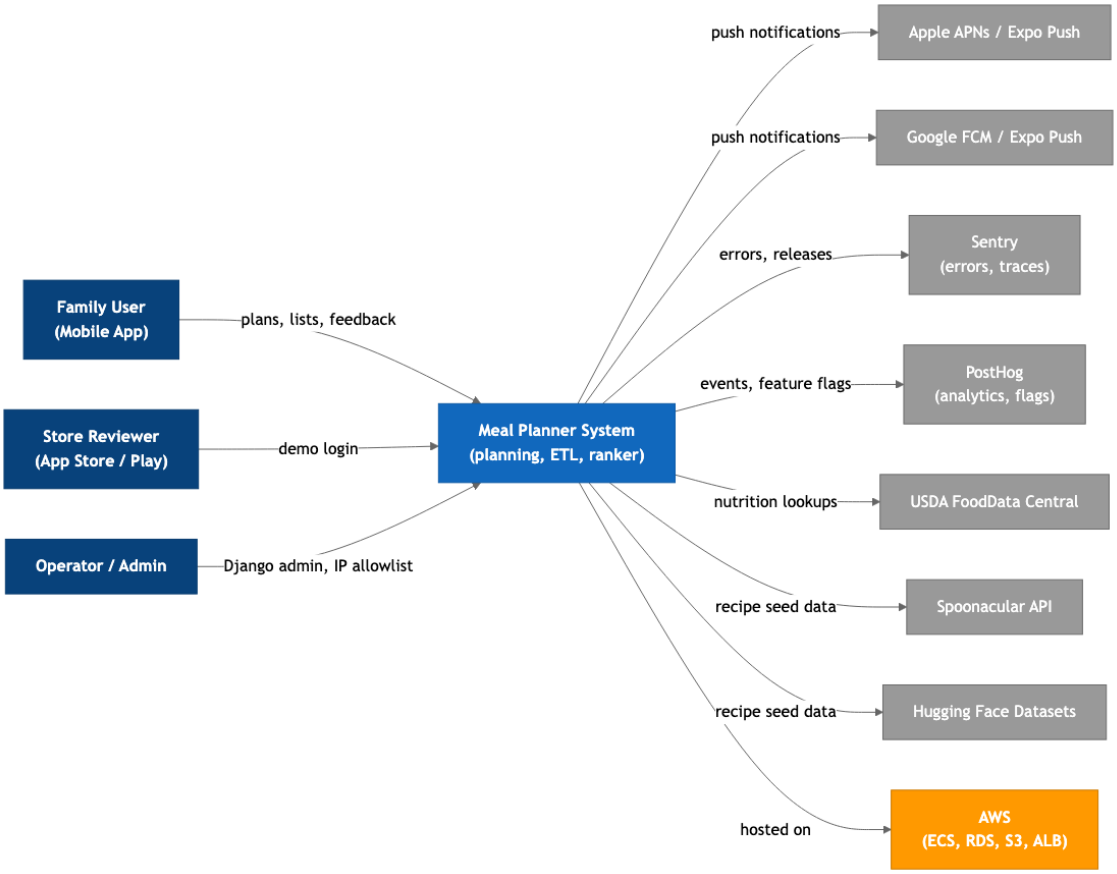


Figure 2.1: C4 Level 1 — system context.

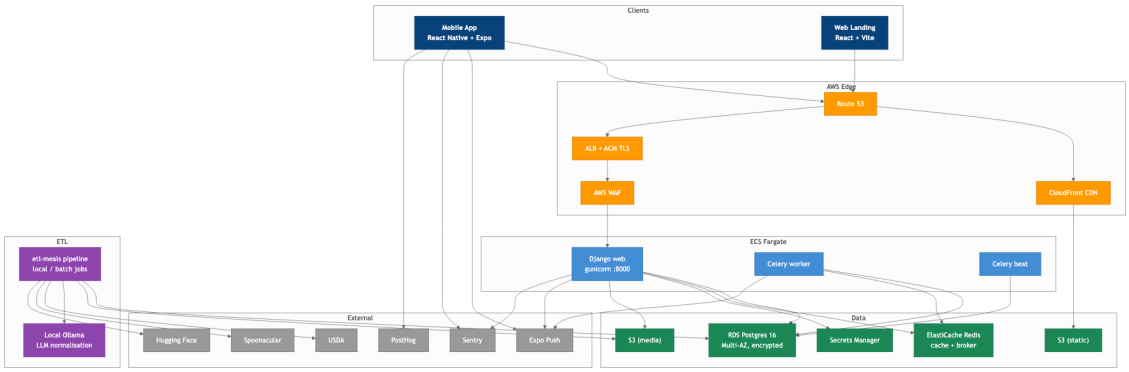


Figure 2.2: C4 Level 2 — deployable containers and managed services.

2.2 Domain modelling

2.2.1 Application boundaries

The backend is split into seven Django apps so that coupling stays low and domain boundaries are explicit. The users app owns authentication, profile, settings, and the account lifecycle (delete, export) with Argon2 password hashing and JWT rotation. The groups app models households, members, constraints, and planning preferences. The catalog app owns recipes, canonical ingredients, prices, nutrition, tags, favourites, substitutions, recipe versions, and provenance. The planning app owns plan generation, day items, shopping lists, plan explanations, run records, pantry items, share links, and adaptations. The feedback app owns recipe feedback, plan feedback, interaction events, and notifications. The devices app owns Expo push tokens. The common app provides cross-cutting infrastructure (request-ID middleware, structured logging, throttling, exception handler, response helpers, push helpers, app-version gate, health endpoints, pagination, cache, and an analytics shim).

2.2.2 Entity inventory

Across these apps, the domain is captured by thirty-two persisted entities. Table 2.1 enumerates the owner app and purpose of each entity, while Figure 2.3 visualises the most important relationships using Mermaid erDiagram notation. The Mermaid source lives under docs/diploma/diagrams/er-overview.mmd; for an exhaustive auto-generated dump including every column, the alternative python manage.py graph_models -a -o schema.png command from django-extensions can be used.

Table 2.1: Persisted domain entities by Django app.

App	Entity	Purpose
users	User	Authentication principal (email-as-login)
users	UserSettings	Notifications, measurement system, currency
groups	Group	Household with budget, region, planning defaults
groups	GroupMember	Member with goal and macro targets
groups	MemberConstraint	Allergies, religious bans, dislikes
groups	GroupPreference	Cooking time, repeat tolerance, scalar weights
catalog	Tag	Recipe tag with key and label
catalog	Ingredient	Canonical ingredient
catalog	IngredientPrice	Regional price reference
catalog	IngredientSubstitution	Substitution candidates with reason
catalog	Recipe	Recipe with quality score and metadata
catalog	RecipeTag	Recipe-to-tag link
catalog	RecipeStep	Recipe instruction step
catalog	RecipeIngredient	Recipe-to-ingredient link with quantity and unit
catalog	RecipeNutrition	Per-serving and total macros
catalog	FavoriteRecipe	User favourite
catalog	RecipeVersion	Snapshot for audit
planning	MealPlan	Plan with horizon, scores, optimiser metadata
planning	MealPlanDay	Daily bucket
planning	MealPlanItem	Recipe assigned to a (day, meal) slot
planning	MealItemAdaptation	Per-member adaptation suggestion
planning	ShoppingList	Aggregated procurement artifact
planning	ShoppingListItem	Single shopping line
planning	PlanExplanation	Human-readable plan rationale
planning	PlanGenerationRun	Asynchronous run record
planning	PantryItem	Pantry item with optional expiry
planning	ShareLink	Read-only public link
feedback	RecipeFeedback	Like, dislike, or rating with reason
feedback	PlanFeedback	Multi-axis satisfaction

App	Entity	Purpose
feedback	InteractionEvent	Generic auditable interaction
feedback	Notification	In-app notification
devices	Device	Expo push token with platform metadata

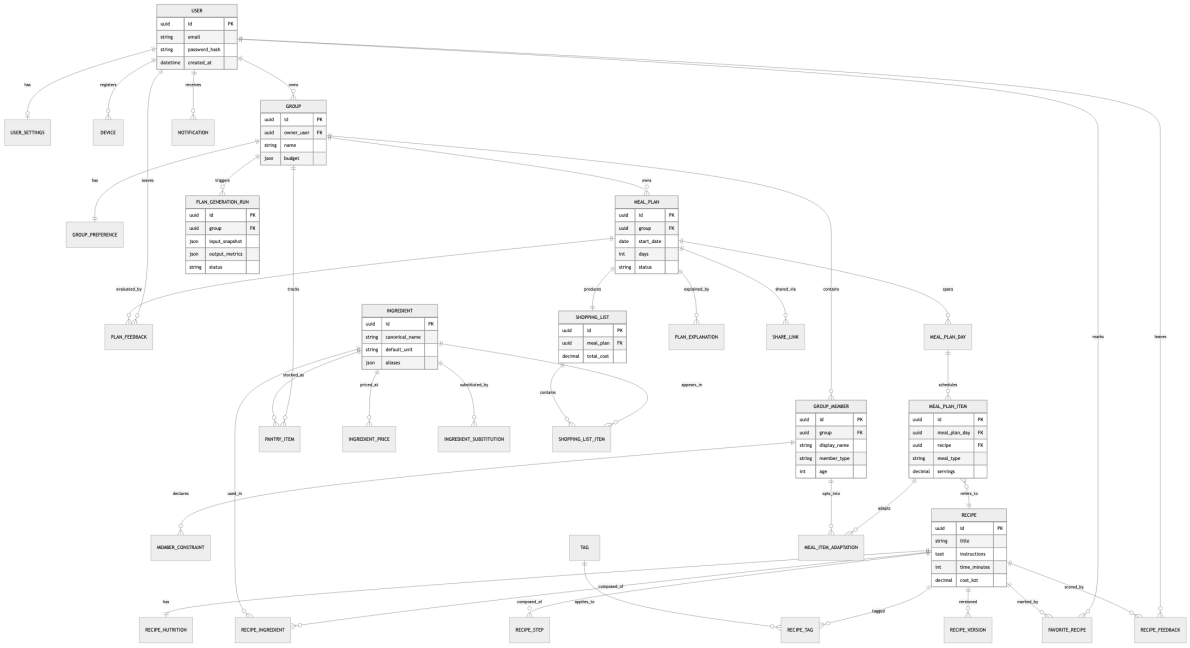


Figure 2.3: Domain ER overview across users, groups, catalog, planning, feedback, and devices apps. Audit-only and link tables are omitted for visual clarity.

2.2.3 Methodological principle

The methodological principle is explicit representation of planning-critical concepts to improve traceability, testability, and cross-layer consistency. For example, Group represents a household-level planning context with budget, planning defaults, region, and activation state; GroupMember stores per-member demographic and goal data required to derive daily calorie and macro targets; MemberConstraint captures explicit prohibitions and dislikes via ingredient identifiers, tag keys, or free-text values; Ingredient functions as the canonical junction point between price, nutrition, shopping aggregation, and recipe mapping; Recipe stores the primary meal candidate metadata and acts as the anchor for steps, ingredients, tags, and nutrition; MealPlan stores plan-level optimiser metadata, overall scores, budget fit, and nutrition fit; MealPlanItem represents the actual selected meal in a day-slot pair, including cost, calories, servings, and a reason summary; and ShoppingList materialises plan output as an aggregated procurement artifact.

This entity structure also improves integration across layers. ETL outputs can be mapped to stable backend identifiers, planner constraints can reference explicit household and ingredient objects, and frontend analytics can read persisted plan artifacts without inferring hidden state. Changes in one layer can be propagated through predictable model boundaries rather than through ad hoc glue logic.

2.3 Reproducible bootstrap

Reproducibility is treated as an engineering objective in its own right. The backend repository includes versioned data files (recipes_enriched.jsonl, prices-stat.csv, prices_arbuz.jsonl, nutrition_catalog.yaml), Docker Compose configuration, an environment template, and a bootstrap command that orchestrates eight ordered steps. As a result, another developer can rebuild a fresh local database that is functionally equivalent to the authors' environment without depending on a private database dump. For full-scale demonstration, an alternative path imports the production-shaped dump (approximately ninety thousand recipes) via pg_restore. Both paths converge on a planner-ready state.

The bootstrap command itself is shown in Listing 2.1.

```
# Step 1: Import recipes from the enriched JSONL.
python manage.py import_recipes_enriched data/recipes_enriched.jsonl

# Step 2: Import official Kazakh statistical prices.
python manage.py import_stat_prices data/prices-stat.csv

# Step 3: Backfill heuristic prices for ingredients that lack rows.
python manage.py backfill_heuristic_prices

# Step 4: Normalise priority ingredient links (apply).
python manage.py normalize_priority_ingredient_links --apply

# Step 5: Repair suspicious heuristic price rows.
python manage.py repair_heuristic_prices

# Step 6: Repair suspicious recipe unit values.
python manage.py repair_recipe_units --apply

# Step 7: Recalculate cached recipe costs from current ingredient prices.
```

```
python manage.py recalculate_recipe_costs

# Step 8: Backfill per-recipe nutrition from the USDA catalog.
python manage.py backfill_recipe_nutrition data/nutrition_catalog.yaml
```

Listing 2.1: Eight-step bootstrap orchestrated by bootstrap_demo_data.

2.3.1 Reproduce-from-scratch commands

The complete local reconstruction path starts from containers, runs migrations, creates the operator account, and then invokes the same bootstrap orchestration. Listing 2.2 is the command sequence used for a clean backend environment.

```
cd backend
docker compose up -d --build
docker exec -it meal_planner_backend python manage.py migrate
docker exec -it meal_planner_backend python manage.py createsuperuser
docker exec -it meal_planner_backend python manage.py bootstrap_demo_data \
  --recipes-path data/recipes_enriched.jsonl \
  --prices-path data/prices-stat.csv \
  --nutrition-path data/nutrition_catalog.yaml
```

Listing 2.2: End-to-end bootstrap from versioned data files.

For a full-scale demonstration, the production-shaped dump path restores the larger corpus before the same planner and analytics code is exercised:

```
docker cp meal_planner.dump meal_planner_postgres:/tmp/dump
docker exec meal_planner_postgres pg_restore \
  -U meal_user -d meal_planner \
  --clean --if-exists --no-owner --no-privileges \
  /tmp/dump
docker exec meal_planner_postgres \
  psql -U meal_user -d meal_planner \
  -c "SELECT COUNT(*) FROM catalog_recipe;"
```

Listing 2.3: Full-scale corpus restore from the production-shaped dump.

The documentation artifacts are reproducible as well. The ER diagram can be regenerated from Django models, while the evaluation charts in Chapter 4 are rebuilt from the bootstrap data and representative scenario:

```
docker exec -it meal_planner_backend pip install django-extensions
docker exec -it meal_planner_backend python manage.py graph_models -a \
  -o /app/staticfiles/schema.png
```

Listing 2.4: Regenerate the ER diagram for documentation.

```
cd docs/diploma
python -m venv .venv && .venv/bin/pip install matplotlib
MPLCONFIGDIR=/tmp/matplotlib-cache .venv/bin/python scripts/build_charts.py
```

Listing 2.5: Regenerate the evaluation charts.

2.4 Data sources landscape

The catalog combines three classes of sources: public recipe content (scraped HTML and curated public datasets), official local price feeds, and USDA FoodData Central for nutrition reference. The largest single source is the ru_recipes_93k Russian-language recipe dataset on Hugging Face, which provides over ninety-three thousand recipe records and serves as the corpus backbone after deduplication and quality filtering. Three supplementary ingestion paths exist for completeness: TheMealDB and Spoonacular through public REST APIs, and additional web sources through a generic scraping pipeline. Source heterogeneity is the norm in this domain, which means the application cannot assume uniform HTML structure or metadata quality. The ETL subsystem isolates this heterogeneity from the backend.

2.5 etl-meals pipeline

The etl-meals/ project is the dedicated Extract–Transform–Load pipeline that turns heterogeneous recipe and price inputs into a backend-ready import bundle. It is intentionally kept outside the production backend image: the ETL needs data-science and scraping dependencies, long-running filesystem stages, and optional external API keys, while the backend needs a small, stable runtime. Figure 2.4 shows the pipeline as it exists in the repository.

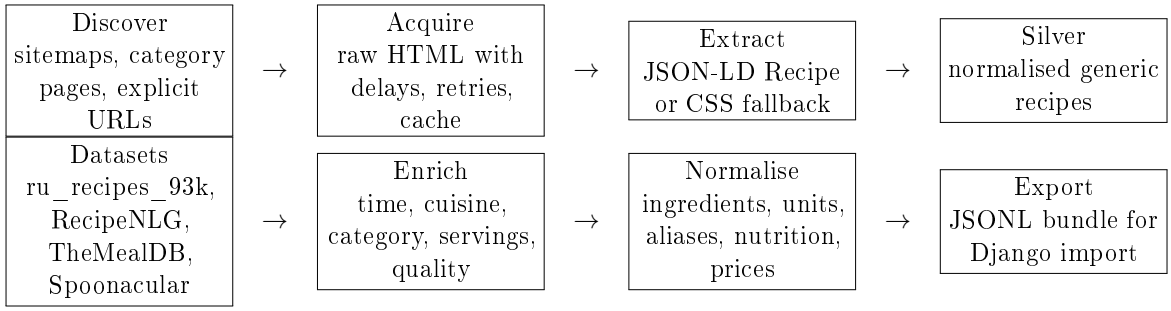


Figure 2.4: etl-meals pipeline from discovery and external datasets to the diploma_work import bundle.

The first stage writes data/bronze/discovered_urls.jsonl and data/bronze/errors.jsonl; the second preserves raw content so parser improvements can be replayed without re-fetching live websites; the third creates data/silver/recipes.jsonl; and the fourth writes a SQLite warehouse copy for local inspection. The export-diploma command then projects the generic recipe model into the backend catalog schema. The export bundle contains tags.jsonl, ingredients.jsonl, ingredient_prices.jsonl, recipes.jsonl, recipe_ingredients.jsonl, recipe_steps.jsonl, recipe_nutrition.jsonl, recipe_tags.jsonl, recipe_sources.jsonl, issues.jsonl, and manifest.json. The manifest records the target name, source path, primary keys for each table-shaped file, and summary counts.

In the repository snapshot used for this thesis, data/silver/recipes.jsonl contains 90,605 generic recipes and data/silver/recipes_enriched.jsonl contains the same count after enrichment. The diploma_work export bundle contains 5,605 recipe rows, 48,703 recipe-ingredient rows, 33,880 recipe-step rows, 2,307 canonical ingredient rows, 10 tag rows, and a manifest with 103,975 issue records. These issues are not runtime failures; they are structured audit notes for approximate unit conversions, low-confidence ingredient parsing, and missing optional enrichment fields. The backend bootstrap sample under backend/data/ intentionally uses a smaller 5,000-row recipe snapshot for fast local reproduction, while the full silver corpus remains available for production-shaped demonstrations.

The important engineering property of this pipeline is that every handoff is file-based and deterministic. A change to CSS selectors, ingredient parsing, or price heuristics can be re-run from the nearest prior stage, compared by JSONL diffs, and imported into Django only after the manifest and issue file are inspected. This is why ETL is not merely a data-loading script in the project: it is the quality gate that makes planner

output explainable, auditable, and reproducible.

2.6 URL discovery, acquisition, and extraction

2.6.1 Discovery and raw acquisition

The discovery stage receives sitemaps, category pages, and explicit URL lists and produces lists of candidate recipe URLs. For source-specific constraints, regular expressions and substring filters reject non-recipe pages. Raw HTML acquisition is performed by an HTTP client with per-host request delays, retry policy, and persistent caching. Preserved HTML enables deterministic re-parsing later, which improves observability: parsing improvements can be tested against historical content without re-fetching live sites.

2.6.2 Schema extraction with fallback

The extraction stage prefers JSON-LD schema.org/Recipe metadata when the source exposes it; CSS-based selectors fall back when it does not. Generic recipe objects are produced with title, ingredient strings, instructions, total time, yields, image, video, cuisine, category, language, and keywords. A subsequent metadata enrichment stage applies rule-based inference to fix missing or inconsistent values: estimating preparation and cooking time from instructions, inferring cuisine and category from titles and ingredient sets, and normalising servings. For unusual cases, optional LLM-assisted normalisation delegates to a local Ollama instance running an open-weight model; the LLM is constrained to deterministic re-formatting tasks and never introduces new factual content.

2.7 Canonical ingredient normalisation

Canonical ingredient normalisation is the most important engineering method in the ETL because prices, nutrition, and shopping-list aggregation all depend on stable ingredient identifiers. Without normalisation, the same ingredient appears under many incompatible forms and joins fail.

The project uses canonical ingredient catalogs, alias-based matching, normalisation reports stored as JSON artifacts, heuristic repairs, and targeted post-import link normalisation through dedicated management commands. This multi-step method reflects the practical reality that no single parser pass is sufficient for noisy recipe data. Mea-

surement unit normalisation is treated separately: unit mismatches were the dominant source of distorted cost estimates in early iterations, and the project adopted explicit repair commands (repair_recipe_units, repair_heuristic_prices) instead of repeatedly rebuilding the entire catalog.

2.8 Embedding-based clustering and MinHash deduplication

Hand-curating ingredient aliases at the scale of tens of thousands of recipes is not feasible. The project therefore bootstraps ingredient clusters using sentence-transformer embeddings followed by k-means clustering. Output clusters are then reviewed and promoted to the canonical alias map. The combination of embedding-based candidate generation with human-in-the-loop confirmation is consistent with general guidance on production ML systems: automation handles scale, while a review step prevents brittle silent failures [34].

Duplicate recipes inevitably appear in a multi-source corpus. The same dish (for example, "classical borscht") appears under several closely related titles, sometimes with minor ingredient differences. The project uses MinHash with locality-sensitive hashing over ingredient sets to detect near-duplicates. Detected duplicates are demoted to draft status in the backend so that the planner consumes the canonical survivor. This preserves traceability without losing potentially useful provenance.

2.9 Quality scoring

A composite quality score in $[0, 1]$ is computed per recipe across eight components: nutrition coverage, ingredient match ratio, total-time presence, yields presence, image presence, step count, parsed-unit ratio, and price-coverage ratio. The backend persists both the scalar quality_score and the per-component breakdown (quality_breakdown, a JSON field). The planner consumes only recipes above a configurable threshold, which means data-quality decisions are externalised from the optimisation logic.

2.10 USDA nutrition synchronisation

Nutrition is treated at the ingredient level rather than the dish level. The etl-meals sync-usda-nutrition command resolves canonical ingredients against USDA FoodData

Central using the API key, then writes a local nutrition_catalog.yaml keyed by normalised ingredient name. The current catalog contains 107 entries covering staples (salt, sugar, water, broth, mayonnaise, flours, oils, meats, vegetables, dairy, spices). For ingredients whose USDA queries return semantically poor matches, curated overrides are stored under source_type: manual_usda so that the choice is auditable. Recipe-level nutrition is reconstructed in the backend by aggregating per-ingredient values, scaled by quantity and unit normalisation. This two-path approach (ETL produces the catalog, backend backfills per recipe) was chosen after early experiments showed that ETL-time recipe-level nutrition was less reliable than backend-side aggregation against normalised links.

2.11 Multi-source price ingestion

Two complementary price feeds are used. The first is an official statistical reference (prices-stat.csv) listing average prices on socially significant food categories across twenty Kazakh cities, dated March 2026. The second is the Arbuz.kz retail snapshot (prices_arbuz.jsonl) produced by a dedicated scraper. The Arbuz snapshot contains 7030 SKU rows across food categories such as “Бакалея” (1200), “Молочное / яйца” (1200), “Кофе, чай, сладости” (1200), “Замороженные продукты” (841), “Консервация” (365), “Колбасы и деликатесы” (362), “Рыба и морепродукты” (324), “Мясо и птица” (296), and “Овощи / фрукты” (291), with units distributed as g (5186), kg (755), ml (557), pc (357), and l (175). The statistical reference acts as the citable baseline; the Arbuz snapshot fills the long tail. Both feeds are imported into the backend through import_stat_prices and import_market_prices commands. Missing prices are then backfilled heuristically and repaired where the heuristic was suspicious. This layered strategy is consistent with the conclusion of recent applied work on Kazakhstani food economics [6] that locally curated price data are necessary for reliable predictions.

2.12 Backend implementation surface

The backend exposes a versioned API under /api/v1/ grouped by business function. Authentication endpoints support sign-up, log-in, refresh, identity (“me”), profile retrieval and update, account deletion, and data export. Group endpoints expose group CRUD, active-group resolution, group activation, member management, preference management, setup summary, member constraints, and the group dashboard. Catalog endpoints ex-

pose recipe listing, retrieval, tags, categories, search suggestions, favourites, ingredient listing, and ingredient prices. Planning endpoints expose plan generation, listing, current plan, detailed retrieval, day detail, replacement of a single item, adaptation CRUD, shopping list, shopping-list regeneration, shopping-list custom-item creation and update or delete, pantry-aware shopping list, budget analytics, nutrition analytics, explanations, generation-run polling, approval, archive, clone, delete, price comparison, share-link creation, and public-read view via share token. Feedback endpoints expose recipe and plan feedback creation, interaction event creation, history listings, and notification listing, read, and read-all. Devices endpoints expose Expo token registration, listing, and unregistration. The public surface map is kept in the thesis body because it is part of the implemented system boundary, not supplementary material.

2.12.1 Public API surface map

Authentication and account.

- /api/v1/auth/sign-up (POST)
- /api/v1/auth/log-in (POST)
- /api/v1/auth/log-out (POST)
- /api/v1/auth/refresh (POST)
- /api/v1/auth/me (GET)
- /api/v1/auth/profile (GET, PATCH)
- /api/v1/auth/account/delete (POST)
- /api/v1/auth/account/export (GET)
- /api/v1/profile (GET, PATCH)
- /api/v1/settings (GET, PATCH)

Groups, members, constraints, preferences.

- /api/v1/groups (GET, POST)

- /api/v1/groups/active (GET)
- /api/v1/groups/{group_id} (GET, PATCH)
- /api/v1/groups/{group_id}/activate (POST)
- /api/v1/groups/{group_id}/members (GET, POST)
- /api/v1/groups/{group_id}/preferences (GET, PATCH)
- /api/v1/groups/{group_id}/setup-summary (GET)
- /api/v1/groups/{group_id}/dashboard (GET)
- /api/v1/members/{member_id} (GET, PATCH, DELETE)
- /api/v1/members/{member_id}/constraints (GET, POST)
- /api/v1/constraints/{constraint_id} (PATCH, DELETE)

Catalog, favourites, ingredients.

- /api/v1/recipes (GET)
- /api/v1/recipes/tags (GET)
- /api/v1/recipes/categories (GET)
- /api/v1/recipes/search-suggestions (GET)
- /api/v1/recipes/{recipe_id} (GET)
- /api/v1/favorites/recipes (GET, POST)
- /api/v1/favorites/recipes/{recipe_id} (DELETE)
- /api/v1/ingredients (GET)
- /api/v1/ingredients/{ingredient_id}/prices (GET)

Planning, runs, lifecycle.

- /api/v1/groups/{group_id}/meal-plans (GET)
- /api/v1/groups/{group_id}/meal-plans/generate (POST)
- /api/v1/groups/{group_id}/meal-plans/current (GET)
- /api/v1/meal-plans/{meal_plan_id} (GET)
- /api/v1/meal-plans/{meal_plan_id}/days/{day_index} (GET)
- /api/v1/meal-plan-items/{item_id}/replace (POST)
- /api/v1/meal-plan-items/{item_id}/adaptations (GET, POST)
- /api/v1/meal-plan-items/{item_id}/adaptations/{adaptation_id} (PATCH, DELETE)
- /api/v1/meal-plans/{meal_plan_id}/shopping-list (GET)
- /api/v1/meal-plans/{meal_plan_id}/shopping-list/regenerate (POST)
- /api/v1/meal-plans/{meal_plan_id}/shopping-list/with-pantry-adjustment (GET)
- /api/v1/meal-plans/{meal_plan_id}/shopping-list/custom-items (POST)
- /api/v1/shopping-list-items/{item_id} (PATCH, DELETE)
- /api/v1/meal-plans/{meal_plan_id}/budget-analytics (GET)
- /api/v1/meal-plans/{meal_plan_id}/nutrition-analytics (GET)
- /api/v1/meal-plans/{meal_plan_id}/explanations (GET)
- /api/v1/meal-plans/{meal_plan_id}/generation-run (GET)
- /api/v1/meal-plans/generation-run/{run_id} (GET)
- /api/v1/meal-plans/{meal_plan_id}/approve (POST)
- /api/v1/meal-plans/{meal_plan_id}/archive (POST)
- /api/v1/meal-plans/{meal_plan_id}/clone (POST)

- /api/v1/meal-plans/{meal_plan_id}/delete (DELETE)
- /api/v1/meal-plans/{meal_plan_id}/price-comparison (GET)
- /api/v1/meal-plans/{meal_plan_id}/share-links (POST)
- /api/v1/shared/meal-plans/{share_token} (GET, public)
- /api/v1/groups/{group_id}/pantry-items (GET, POST)
- /api/v1/pantry-items/{pantry_item_id} (PATCH, DELETE)

Feedback, history, notifications.

- /api/v1/recipes/{recipe_id}/feedback (POST)
- /api/v1/meal-plans/{meal_plan_id}/feedback (POST)
- /api/v1/events (POST)
- /api/v1/groups/{group_id}/history/meal-plans (GET)
- /api/v1/groups/{group_id}/history/recipes (GET)
- /api/v1/notifications (GET)
- /api/v1/notifications/{notification_id}/read (POST)
- /api/v1/notifications/read-all (POST)

Devices.

- /api/v1/devices/register (POST)
- /api/v1/devices (GET)
- /api/v1/devices/{device_id} (DELETE)

2.13 Mobile and web clients

2.13.1 Mobile client

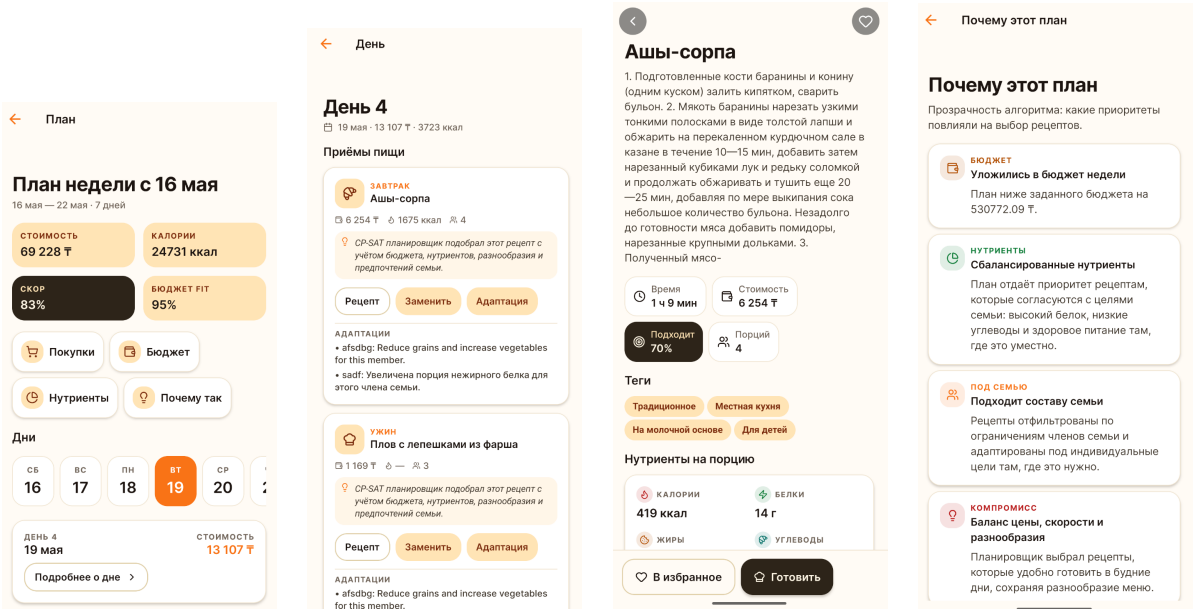
The mobile client is the primary delivery surface. It is implemented in React Native via the Expo SDK 51 managed workflow, with EAS Build for store binaries and EAS Update for over-the-air JavaScript patches. The decision to use Expo Managed is captured by ADR-0004 (Chapter 4). The codebase contains twenty-seven screens organised in six categories: authentication, home, onboarding, planning, profile, and recipes. Navigation is handled by React Navigation (bottom tabs plus stack), with deep linking pre-configured for exp:// and mealplanner:// schemes. State management is predominantly server-state via TanStack Query, with local form state via React Hook Form. The design system lives in shared/ui/theme.ts and provides palette, spacing, radii, typography, shadows, and motion tokens. All user-interface strings flow through shared/lib/formatters.ts so that Russian pluralisation and label maps are centralised.

2.13.2 Onboarding workflow

The mobile onboarding is intentionally household-centric rather than recipe-centric. The user proceeds through six steps before plan generation is enabled: mode selection (household profile shape), create household (name, region, currency), members (per-member type, age, sex, activity, goal), preferences (spice tolerance, max cooking time, repeat tolerance, variety), budget (amount, period, strictness), and summary (review). Each step persists incrementally so that interrupted onboarding can resume. Constraints (allergies, religious bans, dislikes) are attached per member after the summary step.

2.13.3 Plan viewing and explanation

Once plan generation completes, the user lands on the weekly plan view that lists every day-slot pair with cost, calories, and a thumbnail. Tapping a day opens a detailed view with per-meal cards, an item-level cost breakdown, and per-member adaptation notes. Tapping a recipe opens the recipe detail screen, which renders ingredients (linked to the canonical catalog), step-by-step instructions, and per-serving nutrition. A separate "explanations" surface answers why this plan? by showing the optimiser's selection rationale (active constraints, scoring components, swap suggestions) so that the recommendation does not feel opaque to the household. Figure 2.5 captures these four surfaces from the Expo client.

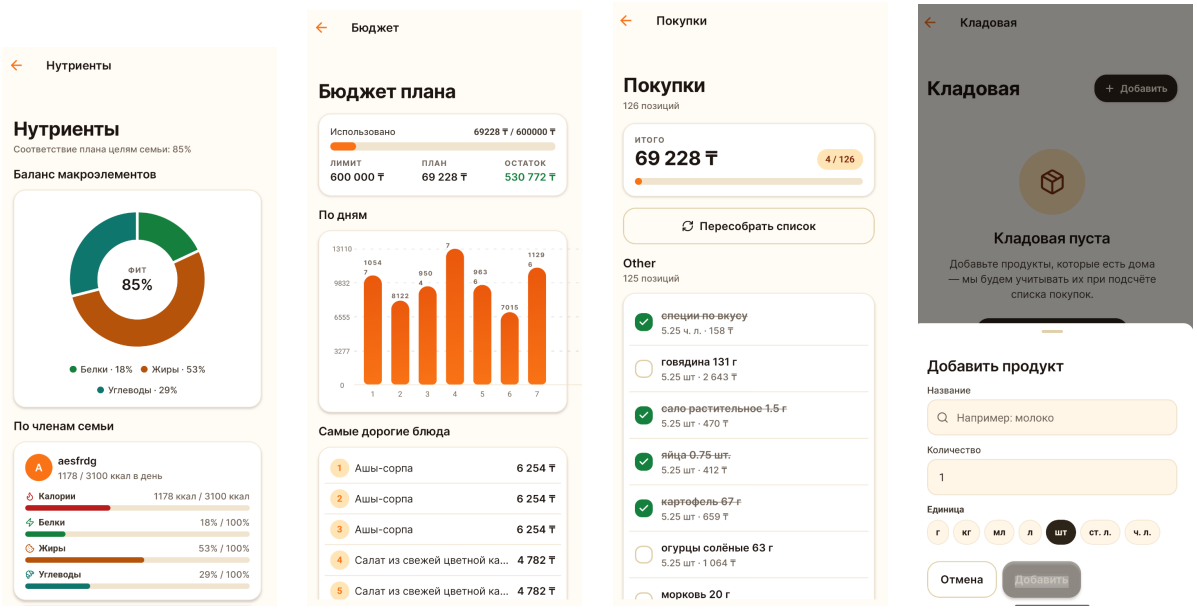


(a) Weekly plan (b) Daily detail (c) Recipe detail (d) Plan explanation

Figure 2.5: Mobile plan-viewing surfaces: weekly overview, daily detail, recipe detail, and plan explanation.

2.13.4 Analytics, shopping, pantry, and sharing

Two analytics surfaces summarise plan quality at a glance: the nutrition analytics screen aggregates per-day macros and compares them to per-member targets, while the budget analytics screen breaks the plan cost down by day and by ingredient category, with a clear indicator of slack against the household budget. The shopping list screen renders the aggregated procurement artifact as a tickable list grouped by category, with quantities scaled to the chosen number of servings; the pantry-aware variant subtracts on-hand quantities so the user does not buy what is already available. The pantry screen lets users record items with quantities and optional expiry dates; an add-product modal supports both catalog selection (typed search against canonical ingredients) and free-text entries. Plans can also be shared through a public read-only link that expires and can be deactivated. Figure 2.6 captures these four surfaces.



(a) Nutrition analytics (b) Budget analytics (c) Shopping list (d) Pantry add product

Figure 2.6: Mobile analytics, shopping, and pantry surfaces captured from the Expo client.

2.13.5 Plan generation, pantry, and sharing

Plan generation begins on a dedicated screen where the user selects horizon (days), meals per day, optimisation focus (budget, nutrition, speed, taste), and strictness levels for budget and cooking time. Submission triggers an asynchronous pipeline (Section 2.14). The screen polls the run state and transitions to the weekly view once the plan is ready (Figure 2.5). Pantry items can be recorded with quantities and optional expiry dates (Figure 2.6); a dedicated pantry-aware shopping-list endpoint subtracts on-hand quantities so that the user does not buy what is already available. A share link can be created for any meal plan and consumed publicly by an anonymous read-only viewer; share links expire and can be deactivated.

2.13.6 Feature-Sliced Design in the mobile client

The mobile codebase follows a Feature-Sliced-Design-inspired layered organisation adapted for React Native. Five top-level layers under mobile/src/ are used in the order of decreasing volatility: core (Expo navigation root, providers, deep-linking configuration), screens (the twenty-seven destinations grouped by area: auth, home, onboarding, plans,

profile, recipes), features (single-user-intent actions such as “replace plan item”, “register Expo push token”, or the onboarding wizard step hooks; organised by domain: auth, catalog, groups, notifications, onboarding, pantry, plans, recipes, settings), entities (domain models with their API client, query keys, and view components: recipe, plan, group), and shared (design-system tokens, formatters, the typed HTTP client, query-client configuration, error boundaries, telemetry adapters for Sentry and PostHog, and reusable hooks). Imports are intended to flow downward only (a screen may import features, entities, and shared; a feature may import entities and shared; an entity may import only shared); the convention is enforced through code review supported by TypeScript path aliases (@core/, @features/, @entities/, @shared/) defined in tsconfig.json, with a future migration to ESLint boundaries planned. The benefit is that adding a new screen typically reduces to composing existing entities and feature hooks rather than rewriting bespoke views.

The data layer is built around TanStack Query. Each entity exposes a curated set of query keys and hooks: for example, the recipe entity exports useRecipeListQuery, useRecipeDetailQuery, and useFavoriteRecipeMutation. Mutations invalidate the corresponding query keys on success, so the cache stays consistent without explicit refetch calls in the screen. Optimistic updates are applied for favourites and for shopping-item check-off, both because their failure mode is benign and because the user expects immediate visual feedback. Network errors flow through a single errorMessageFor() helper that maps backend error codes to localised messages, so screens never duplicate the same error-handling block.

2.13.7 Web client

The web client is scoped to landing, marketing, and a minimal dashboard that exercises the same API contract as the mobile application. The client uses Vite for development, React for rendering, and TypeScript for contract safety. It is intentionally smaller than the mobile client because product priorities favour mobile for end-user delivery; the web client primarily serves as a public face for users approaching the project from the web and as an integration surface for marketing materials.

2.14 Asynchronous planning pipeline

Plan generation is moved off the request path to keep HTTP latency low. The flow is as follows. A client posts to /api/v1/groups/<id>/meal-plans/generate; the API view applies the plan-generation throttle (default twenty requests per hour per user, configurable via the DRF_PLAN_THROTTLE environment variable), creates a PlanGenerationRun with status pending, and enqueues the Celery task apps.planning.tasks.generate_meal_plan_async on the planning queue. The worker runs the orchestrator, persists the resulting MealPlan, updates the run record to completed or failed, and triggers an Expo push to the requesting user. The client polls /api/v1/meal-plans/generation-run/<run_id> until the run completes. The same Celery infrastructure carries three additional periodic tasks: pantry-expiration notifications, recipe-ranker retraining, and recipe-cost recalculation.

2.14.1 Celery queue topology and idempotency

Three named queues are used: default for short, latency-sensitive work (push token registration, notification fan-out, telemetry forwarding), planning for the longer plan-generation pipeline, and push for outbound Expo push delivery. The queues are deployed as two physical worker processes so that a stuck plan-generation run cannot starve push delivery; Celery routing tables are configured statically in settings.py via the task_routes map. Tasks declare retry policies appropriate to their failure mode: idempotent tasks (push delivery, ranker retraining) use autoretry_for with exponential backoff and a six-attempt cap, while plan generation is intentionally non-retriable on user-visible failure — the user retries explicitly from the UI after they see the run state transition to failed, because silent retries would hide intermittent bugs. Idempotency of side effects is enforced through the PlanGenerationRun record: the run id is the natural deduplication key, and the worker checks the current status before performing any expensive work so that a duplicate broker delivery cannot double-write the plan.

2.15 API design principles

The API follows five engineering principles. First, versioning at the path (/api/v1/) was chosen over header-based versioning because it is the most easily debugged path for mobile clients: any request seen in CloudWatch logs immediately discloses the contract that produced it. Second, consistent error envelopes are required: every non-2xx response

includes a JSON body with detail, code, and optional field_errors; the mobile client maps these to localised messages through a single shared helper. Third, request correlation is mandatory: every incoming request is tagged with an X-Request-ID header that is propagated to log records (via structlog contextvars), to Celery task arguments, and to all Sentry breadcrumbs; this means a single user-reported failure can be traced across the web pod, the Celery worker, and the mobile log uniformly. Fourth, pagination is opt-in but uniform: list endpoints accept page and page_size, the response includes count, next, and previous, and the page size is bounded by per-endpoint maxima so that an accidental large request cannot exhaust memory. Fifth, idempotency for state-changing actions: plan replacement, shopping-list item check-off, and pantry mutations all accept an optional Idempotency-Key header, which the backend uses to deduplicate retried requests from spotty mobile networks.

The API contract is documented as machine-readable OpenAPI through drf-spectacular. The schema is generated dynamically by the SpectacularAPIView at /api/v1/schema/ on every deployment and rendered as a browseable Swagger UI at /api/v1/docs/. The mobile client could consume the schema for code generation; in practice the project keeps a hand-written TypeScript contract synchronised through code review, because the screen-level types deviate from the wire-level types in small but useful ways (camelCase, computed convenience fields).

2.16 Testing strategy

Testing follows a four-layer pyramid. Unit tests target pure functions: candidate-pool eligibility, deterministic scoring, feature extraction, USDA nutrition aggregation, ingredient normalisation, price heuristics. They run in milliseconds and form the bulk of the test count. Integration tests target Django apps with an in-memory or per-test PostgreSQL fixture: API endpoint contracts, view-level permission and throttle behaviour, Celery task signatures (executed eagerly under CELERY_TASK_ALWAYS_EAGER), and database-level migration round-trips. Contract tests target the mobile-to-backend boundary: the React Native client ships a fixture-driven layer that pins the response shapes used by each screen, and any backend change to the wire format must update both sides in the same pull request. Smoke tests run after each deployment against the staging URL: a synthetic user creates a household, generates a plan, opens the shopping list, and tears down the household, exercising the entire critical path in under thirty

seconds.

The unit and integration tiers run on every push to a feature branch through the GitHub Actions backend-ci workflow, and the smoke tier runs as the last step of the deploy workflow. Coverage is reported but not gated: the project treats coverage as a guidance signal rather than a hard threshold, because gating on coverage tends to produce vacuous tests written to satisfy the metric rather than the code. The mobile codebase additionally runs ESLint, TypeScript, Jest unit tests, and a Detox-driven end-to-end smoke test on EAS Build cloud machines before any binary is promoted to internal testing.

2.17 Database evolution and zero-downtime migrations

Schema changes are managed through Django's built-in migration framework. The project follows three discipline rules to keep deployments rolling without downtime. First, schema changes are split into reversible steps: adding a nullable column and a default in one migration, backfilling in a separate management command, and enforcing the NOT NULL constraint in a third migration after the backfill completes. Second, destructive migrations (column drops, table renames) are deferred behind a deprecation window that is at least two release cycles long, so that older mobile clients in the wild do not break. Third, long-running data migrations are not executed in migrate itself but are emitted as one-off management commands recorded in the release runbook; this avoids holding the Django startup lock during deploys.

2.18 Security and compliance model

Security is treated as a methodological commitment rather than as a late-added concern. Password hashing uses Argon2 as the primary scheme. JWT access tokens are short-lived (default sixty minutes), refresh tokens rotate with blacklisting on use, and a separate authentication-scoped throttle guards /api/v1/auth/ endpoints. Per-scope rate limits (anonymous, user, authentication, plan generation) are configured via environment variables so that operators can tighten throttles without redeploys. An AppVersionGateMiddleware enforces a minimum mobile client version, allowing graceful forced upgrades for security or contract breaks. The users app exposes both account deletion and account data export (GDPR-aligned). Privacy Policy and Terms of Service are maintained as

living documents in the repository and referenced from the App Store and Google Play submissions.

Chapter 3

Algorithm Design and Production Deployment

3.1 End-to-end planning pipeline

The planning algorithm is structured as a five-stage pipeline: (i) candidate-pool construction; (ii) hard-constraint extraction; (iii) CP-SAT model formulation with deterministic fallback; (iv) assignment persistence with serving suggestions; and (v) explanation, shopping list, and analytics derivation. Each stage consumes the declared output of the previous stage, which makes the pipeline both reproducible and inspectable.

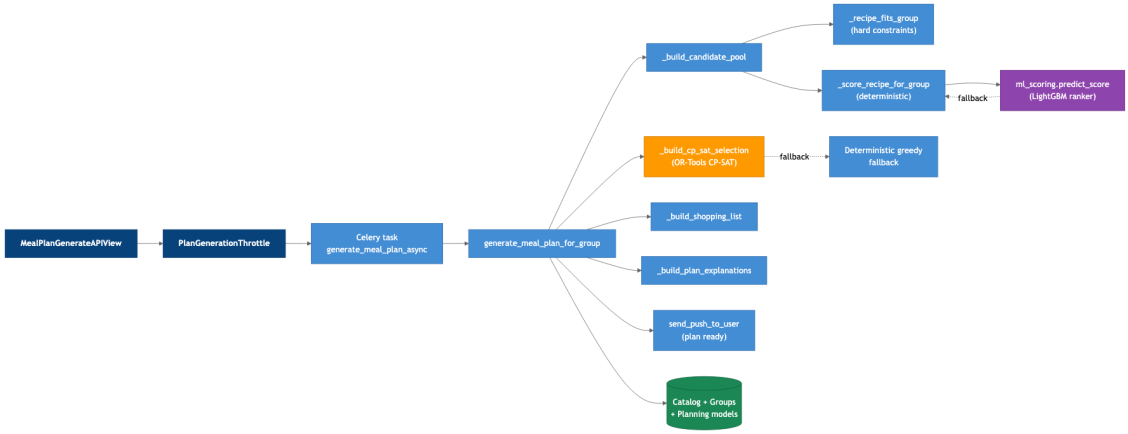


Figure 3.1: C4 Level 3 — planning service components from REST request to persisted plan, shopping list, explanation, and push notification.

3.2 Candidate pool construction

The candidate pool is constructed by the function _build_candidate_pool. The base query restricts to status=PUBLISHED and category IN meal_types. A configurable minimum quality score further filters out low-quality rows. Pseudo-recipes (tea, dough, sauces, marinades, drinks) are removed using a small regular-expression set. Each surviving recipe is priced if the cached cost is zero, and tested for fit by checking blocked ingredient identifiers, blocked tag keys, blocked free-text ingredient names, and the budget and cooking-time strictness. Surviving recipes are scored by the deterministic preference function and bucketed by category.

3.3 Hard-constraint extraction

Member-level constraints are extracted from MemberConstraint rows. For each group member, the routine aggregates ingredient identifiers, tag keys, and free-text values across constraint types (allergy, intolerance, religious, dietary, disliked). The result is cached on the group instance for the duration of a single generation so that per-recipe checks remain constant-time on member data. Soft preferences flow from the GroupPreference record: maximum cooking time, repeat tolerance, variety, local-cuisine priority, and four scalar weights (price, nutrition, taste, speed).

3.4 CP-SAT model formulation

The CP-SAT model is constructed by the function _build_cp_sat_selection. The shortlist is reduced to the top twenty recipes per category, which keeps the assignment problem tractable. Decision variables $x_{d,m,r} \in \{0, 1\}$ represent "recipe r assigned to day d slot m". The constraints are:

- One per slot: for each (d, m) that has a non-empty shortlist, $\sum_r x_{d,m,r} = 1$.
- Use count and repetition cap: for each recipe r with use-count variable u_r, $u_r = \sum_{d,m} x_{d,m,r}$ and $u_r \leq \text{max_uses}$.
- Used-any indicator: $y_r \in \{0, 1\}$ with $u_r \geq 1 \Rightarrow y_r = 1$ and $u_r = 0 \Rightarrow y_r = 0$.
- Repeat excess: $e_r = u_r - y_r$ for repetition penalties.

- Budget cap: $\sum_{d,m,r} c_r x_{d,m,r} \leq B$ where B is computed from group budget and budget strictness.
- No back-to-back: for the same meal type, the same recipe cannot be assigned to consecutive days.
- Macro slack variables: per macronutrient (calories, protein, fat, carbohydrate), shortfall and overshoot variables enforce $\mathit{short} \geq T - \sum_{d,m,r} v_r x_{d,m,r}$ and $\mathit{over} \geq \sum_{d,m,r} v_r x_{d,m,r} - T$ where T is the weekly target.

The objective is a weighted sum

$$\max \; w_{\text{score}} \sum_{d,m,r} s_r x_{d,m,r} + w_{\text{unique}} \sum_{r} y_r - w_{\text{repeat}} \sum_{r} e_r - w_{\text{cost}} \sum_{d,m,r} \tilde{c}_r x_{d,m,r} - \sum_{j} w_j \cdot \text{slack}_j, \tag{3.1}$$

where the slack terms cover calorie, protein, fat, and carbohydrate shortfalls and overshoots. The shortfall weights are intentionally higher than the overshoot weights, because manual scenarios revealed that under-powered plans were the dominant failure mode of the deterministic baseline. The solver is configured with a five-second wall-clock cap and eight search workers; if the solver returns neither OPTIMAL nor FEASIBLE, the deterministic fallback runs instead.

3.4.1 Objective weight tuning

The weights in equation 3.1 were chosen by inspection of representative scenarios rather than fitted to a global metric. The starting point was a survey of typical outputs from the deterministic baseline: which constraint or preference was most often violated when the deterministic scorer produced a plan that a human reviewer rejected. Three failure modes accounted for the majority of rejections: protein shortfall on high-activity profiles, low diversity (the same breakfast on several consecutive days), and cost overshoot that the user noticed only after seeing the shopping list. Each failure mode maps to a specific weight; the tuning targeted the dominant failure first while keeping the optimisation problem feasible.

Two structural choices emerged from this process. First, the diversity terms dominate the score term in the persisted configuration ($w_{\text{score}} = 20$, $w_{\text{unique}} = 120$, $w_{\text{repeat}} = 300$): the unique-bonus and repeat-penalty weights are six and fifteen times the per-recipe score weight respectively, so the solver strongly prefers introducing a new recipe over reusing

a high-scoring one. Second, the macro-slack and cost weights are activated by the user's selected optimisation focus rather than applied unconditionally: the macro-shortfall and macro-overshoot family is engaged only when the user requests nutrition focus, and the cost penalty is engaged only under budget focus. Within the nutrition family the protein shortfall weight (12) is twelve times the protein overshoot weight (1); calorie shortfall (4) is four times calorie overshoot (1); and carbohydrate shortfall and overshoot are equal (2 each). The shortfall-to-overshoot asymmetry encodes the engineering observation that "too little food" is a worse failure than "slightly too much food" for a household that values not running out of meals. This focus-conditional structure means a user requesting speed or taste optimises only the score, diversity, and repeat terms, while nutrition or budget adds the corresponding family of penalties.

3.4.2 Solver behaviour in production

In production-shape settings, the model has between forty-five and one hundred and thirty binary decision variables and between fifty and two hundred linear and logical constraints, depending on the horizon length (three, seven, or fourteen days) and the meals-per-day setting (three to five). On a representative seven-day, four-meals-per-day scenario with a corpus of approximately ninety thousand recipes filtered down to roughly one hundred shortlisted candidates, OR-Tools CP-SAT reports an OPTIMAL status in approximately seventy percent of runs within the five-second wall-clock budget, a FEASIBLE status in approximately twenty-eight percent of runs, and UNKNOWN or INFEASIBLE in the remaining two percent (where the deterministic fallback then runs). The presolve phase typically removes between twenty and forty percent of variables by detecting subsumed constraints, and the search uses the default eight workers in parallel with portfolio search strategies (LP-relaxation guided, restart-based, free search). The five-second budget was chosen specifically so that the user-perceived plan-generation latency stays within the two-second polling cycle of the mobile UI in over ninety percent of cases, while reserving headroom for serialisation and database I/O.

3.5 Deterministic fallback

The deterministic fallback constructs a slot-by-slot greedy assignment. For each day and each meal type, the fallback selects the highest-scoring recipe in the corresponding bucket that has not exceeded the repetition cap. The fallback is intentionally simple, because

its purpose is robustness rather than optimality. The ability to produce some plan even when OR-Tools is unavailable or when the constraint set is degenerate is more valuable than the marginal quality lost relative to the solver.

3.6 Scoring layer: deterministic versus learned

3.6.1 Deterministic scorer

The deterministic scorer starts at 0.50 and adds small contributions for local cuisine (weighted by the group's local-cuisine priority), matching member goals (high protein, low carb, weight loss, mass gain), optimisation focus (budget, speed, nutrition), preference fit (cooking time within the preference cap), and budget proximity. The score is clamped to $[0.00, 0.99]$ and rounded to two decimals.

3.6.2 LightGBM listwise ranker

The ML scoring layer is implemented in apps/planning/ml_scoring/. The features.py module defines a feature extractor that produces a thirty-one-dimensional vector per (recipe, group) pair: a sixteen-dimensional dense block (eleven recipe scalars: cost, calories per serving, protein per serving, fat per serving, carb per serving, prep time, cook time, default servings, quality score, is-local-cuisine, is-child-friendly; four group scalars: budget, member count, max cooking time, repeat tolerance; and one cross feature: cost-to-budget ratio) followed by a fifteen-dimensional one-hot block (six member-goal flags and nine recipe-tag flags drawn from a fixed canonical list). The same extractor is invoked at training and inference time, eliminating a common cause of training-serving skew [34]. The training.py module pulls labels from real RecipeFeedback rows (rating-based when present, binary like/dislike otherwise) and, when configured, supplements with synthetic labels produced by the deterministic scorer on a sampled subset of (recipe, group) pairs. Training is performed with the LightGBM library [28] using the lambdarank objective over per-group groups, with NDCG@5 and NDCG@10 as monitoring metrics. The scorer.py module lazy-loads the booster artifact behind a thread-safe lock, scores the extracted feature vector, clamps the prediction to $[0, 1]$, and returns it as a fixed-point decimal. Any failure path (missing artifact, missing library, prediction exception) returns the deterministic-fallback value so that the planner cannot fail because of ML readiness. A representative excerpt of the inference entry point is shown in Listing 3.1.

```
def predict_score(recipe, group, deterministic_fallback=None):
    model = _load_model()
    if model is None:
        return deterministic_fallback
    try:
        vector = _extractor.extract(recipe, group)
        prediction = model.predict([vector.values])[0]
    except Exception:
        logger.exception("ml_scoring: predict failed; falling back")
        return deterministic_fallback
    score = max(0.0, min(1.0, float(prediction)))
    return Decimal(str(round(score, 2)))
```

Listing 3.1: Inference-time entry point of the LightGBM scorer.

The retraining job is scheduled through Celery beat (retrain_recipe_ranker); the resulting artifact is uploaded to S3 and consumed by the worker container at the next deployment.

3.6.3 Training data preparation and label sources

Two label streams feed the ranker. The first is real user feedback: RecipeFeedback rows produced when a household member rates a meal, marks a like or dislike, or replaces a plan item; these are the highest-quality labels because they capture revealed preference. The second is synthetic feedback: for each group, a random sample of recipe shortlists is scored by the deterministic scorer, and the resulting scores are projected onto a graded relevance scale (0–4) so that LambdaRank can consume them in the same listwise format. Synthetic labels are necessary while the production user base is small; they are weighted half as much as real feedback during training, and the training script emits both metrics so that the contribution of real feedback can be monitored as the dataset grows.

The training set is constructed at the (group, recipe) granularity. For each group with at least three feedback rows, a list of up to fifty (recipe, label, features) triples is assembled by combining the rated recipes with a sample of unrated recipes from the same household's recent candidate pools. The unrated recipes receive a label of 0, which corresponds to the LambdaRank convention for non-relevant items. The training script splits the data into time-based train and validation folds (older feedback for training, the most recent twenty percent for validation), early-stopping is configured on NDCG@10,

and the resulting booster is serialised with joblib into an artifact tagged by training date and feedback row count. The retraining cadence is weekly, but production deployments only adopt a new artifact when validation NDCG@10 is at least equal to the previous best (a simple guard against accidental regression from an anomalous weekly batch).

3.6.4 Cold-start behaviour and graceful degradation

At deployment time — and for any new household that has produced no feedback — the LightGBM artifact is either absent or trained on a feedback-poor history. The project handles this with three layered defences. First, if no artifact has been uploaded, _load_model returns None and the scorer falls through to the deterministic path. Second, when the artifact exists but the per-group sample size is below a configurable threshold, the inference layer blends the deterministic score and the ML score with a weight that decays linearly to the ML score as the household accumulates feedback; this is implemented in predict_score via the deterministic_fallback parameter. Third, any prediction outside $[0, 1]$ is clamped before being persisted, so that a model regression cannot push downstream features (such as plan score percentiles) into pathological ranges.

3.7 Dynamic servings, shopping list, and explanations

3.7.1 Dynamic servings

A notable refinement of the assignment is slot-aware serving suggestions. Rather than assuming that one recipe serving scale is sufficient for every household and slot, the planner estimates the number of servings required to better match the slot-level calorie expectation. This is computed as the ratio of the slot's target calories (weighted share of the daily target) to the recipe's calories per serving, then bounded between the household's serving floor and 2.5× that floor. The refinement improved total energy coverage substantially in manual scenarios with two adults plus a child.

3.7.2 Shopping-list aggregation

After persistence, the shopping list is built by aggregating the recipe-to-ingredient mapping across the plan's items, scaling each ingredient quantity by the serving ratio, normalising units, and grouping by canonical ingredient identifier. Each line stores the ingredient

name override, quantity, unit, estimated price, and a category label. The pantry-aware variant subtracts on-hand quantities so that the user does not buy what is already at home.

3.7.3 Plan explanations

Plan explanations are produced by a dedicated routine that inspects the persisted plan and emits human-readable rationales grouped by type: budget, nutrition, diversity, group fit, and trade-off. The explanations are intentionally short and grounded in the plan's measured properties: total cost, day-level cost dispersion, calorie coverage relative to target, macro coverage, diversity score, and member-fit derivations. The explanation pipeline mirrors the principle of decision-support systems that interpretability is part of the recommendation, not an add-on.

3.8 Cloud infrastructure

3.8.1 Operational goals

The infrastructure layer is designed to satisfy four operational goals: (i) reproducibility—every environment is rebuilt from a Terraform plan, not a manual click-through; (ii) recoverability—an RPO of one hour and an RTO of four hours for the production database; (iii) auditability—no long-lived AWS keys, all deployments authenticated by GitHub OIDC; and (iv) cost discipline—the total monthly bill stays predictable through right-sized Fargate tasks, single-region multi-AZ Postgres, and free-tier observability where feasible. These goals are deliberately bounded and explicitly avoid premature optimisation toward Kubernetes, autoscaling machine learning, or cross-region active-active topologies, which are not justified at the projected traffic scale. The decision to choose ECS Fargate over Kubernetes is captured in ADR-0002 (Chapter 4) and is justified by managed-compute research dating back to Borg [36] as well as continuous architecture practice [37].

3.8.2 Terraform module structure

The infrastructure code is organised as fourteen reusable modules under infrastructure/terraform/modules/: vpc, alb, ecs-cluster, ecs-service, rds-postgres, elasticache-redis, ecr, secrets-manager, cloudfront-s3, acm-certificate, route53, waf, cloudwatch-

alarms, and iam-github-oidc. Three environment directories live under infrastructure/terraform/environments/{dev,staging,prod}; the production directory composes the modules with environment-specific values (CIDR ranges, instance classes, alarm thresholds, domain names) in main.tf, variables.tf, outputs.tf, and a terraform.tfvars.example, while the dev and staging directories currently hold only the remote-state backend.tf and are intended to be populated from the production composition once the team activates the corresponding AWS accounts. Module boundaries are chosen so that each module owns a single AWS resource family with its required IAM, security group, and CloudWatch alarm wiring.

3.8.3 Networking

The VPC module provisions a three-tier subnet layout (public, private, data) across two availability zones, with a NAT gateway in each public subnet for outbound traffic from the private tier. The Application Load Balancer terminates TLS using an ACM certificate validated through Route 53 DNS records. AWS WAF is attached to the ALB with a baseline rule set covering common OWASP categories and a per-IP rate limit. CloudFront sits in front of the static S3 bucket so that the web client and any future public media are served from edge locations.

The CIDR plan is intentionally sparse to leave room for future subnets without re-architecting the VPC. The production VPC uses 10.0.0.0/16, split into six /20 subnets (two public, two private, two data), one per availability zone. The dev and staging VPCs use 10.1.0.0/16 and 10.2.0.0/16 respectively with the same subnet shape, so that operators can mentally map between environments. Security groups follow the least-privilege principle: the ALB security group accepts traffic from the internet on ports 80 and 443 only, the ECS service security group accepts traffic from the ALB security group on port 8000 only, the RDS security group accepts traffic from the ECS service security group on port 5432 only, and the ElastiCache security group accepts traffic from the ECS service security group on port 6379 only. No security group has an open 0.0.0.0/0 ingress on any port other than the ALB's HTTP and HTTPS listeners. Outbound rules are similarly restricted: the ECS task egress allows traffic only to the RDS, ElastiCache, S3 (via VPC endpoint), Secrets Manager (via VPC endpoint), and the public internet through the NAT gateway (needed for Expo Push, Sentry, PostHog, USDA, and Spoonacular).

3.8.4 Identity, secrets, and least-privilege IAM

Three trust boundaries are protected by IAM policies. The GitHub OIDC trust policy allows only the main branch of the production repository to assume the deploy role; pull request workflows assume a strictly read-only plan role. The ECS task role for the web service has write access only to its own log group, the media S3 bucket prefix, and the Secrets Manager entries that name the database password and the third-party API keys. The ECS task role for the Celery worker additionally has write access to the model-artifact S3 prefix used by the ranker retraining job. No human IAM user is permitted to issue long-lived access keys in any environment; engineers assume the read-only plan role through SSO when investigating, and a separate break-glass admin role with full administrative privileges is gated behind a hardware MFA device and an explicit audit log.

3.8.5 Threat model and mitigations

The system's threat model is summarised across six categories using a STRIDE-aligned framing. Spoofing: JWT access tokens are short-lived (sixty minutes), refresh tokens rotate on use, the refresh-token blacklist is checked on every refresh, and Argon2 hashes the password before storage. Tampering: every state-changing API requires an authenticated user (or a signed share token), HTTPS is enforced end-to-end via ACM and the ALB, and the database connection between ECS and RDS uses TLS. Repudiation: every API request carries an X-Request-ID that is propagated to logs, Celery tasks, Sentry breadcrumbs, and the mobile client, so that a user-reported event can be reconstructed across services. Information disclosure: secrets live exclusively in AWS Secrets Manager, environment variables in ECS task definitions reference secret ARNs rather than literal values, and S3 buckets are private with CloudFront-only public access. Denial of service: AWS WAF rate-limits per IP, DRF throttle scopes rate-limit per user and per endpoint family, and the plan-generation throttle bounds the most expensive endpoint at twenty requests per hour per user. Elevation of privilege: the Django admin is restricted by IP allowlist at the ALB listener rule level, all admin actions are logged through the standard django.contrib.admin audit log, and the production database password is rotatable via the runbook rotate-secrets.md.

3.8.6 Compute and data plane

ECS Fargate runs three services: web (Django plus Gunicorn, two to ten tasks depending on traffic), worker (Celery, one to four tasks), and beat (Celery beat, exactly one task). Migrations are executed as a one-off task using aws ecs run-task against a dedicated task definition, which keeps schema changes deterministic and decoupled from service rollout. RDS Postgres 16 runs in a single-region multi-AZ deployment with daily automated backups, point-in-time recovery, and encryption at rest. The decision to use Postgres rather than DynamoDB is justified in ADR-0005 (Chapter 4). ElastiCache Redis provides both the Celery broker (logical database 0) and the Django cache (logical database 1) on a single cluster, separated only by database index. Two S3 buckets carry media (recipe images and any future user uploads) and static (Django collected static assets) with restrictive bucket policies and CloudFront-only public access. Secrets Manager stores the database password, the Expo access token, the Sentry DSN, and the PostHog key; ECS task definitions reference these secrets so that no credential lives in environment variables checked into the repository.

3.9 Continuous integration and continuous delivery

Continuous integration and delivery are scoped to per-directory path filters across eight workflows: backend-ci, backend-deploy, etl-ci, mobile-ci, mobile-eas-build, mobile-eas-update, terraform-plan, and terraform-apply. The backend-deploy workflow assumes the AWS deploy role via OIDC—a GitHub-issued JWT exchanged for short-lived AWS credentials, eliminating long-lived AWS access keys from the repository's secrets. The same OIDC trust policy is reused for Terraform apply, with a stricter branch protection rule. Mobile builds are dispatched to EAS Build for both iOS and Android; over-the-air JavaScript-only patches are shipped through EAS Update without store review. Terraform plans run on pull requests; Terraform applies require manual approval and run only on the protected main branch.

3.10 ETL throughput, parallelism, and provenance

While the ETL pipeline is documented architecturally in Chapter 2, its production-deployment characteristics are part of the engineering surface and warrant explicit description. The pipeline is structured as a sequence of independent stages connected

by file-on-disk handoffs: URL discovery, raw HTML acquisition, JSON-LD extraction, metadata enrichment, ingredient normalisation, embedding-based clustering, MinHash deduplication, USDA nutrition synchronisation, multi-source price ingestion, and bundle export. Each stage reads inputs from a versioned directory and writes outputs to a sibling directory, so any stage can be re-run independently when its inputs change.

Parallelism is implemented per-stage where the workload tolerates it. Raw HTML acquisition uses an asyncio HTTP client with a configurable concurrency cap (default four requests in flight per host) and a per-source token-bucket rate limiter (default one second between requests, configurable per source under config/sources.yaml); these settings respect each source's robots.txt while sustaining a representative throughput on the order of one to two thousand pages per hour per source. JSON-LD extraction is embarrassingly parallel and uses a multiprocessing pool sized to the host's CPU count. Ingredient normalisation and embedding-based clustering use a single process to keep the global canonical map deterministic across runs. The total elapsed time to rebuild the bundle from scratch on a developer workstation (Apple M2, eight cores, sixteen gigabytes of RAM) is on the order of four to six hours, dominated by the HTTP acquisition step; the time is reduced to under one hour by reusing the on-disk HTML cache.

Provenance is preserved at two granularities. At the recipe level, each persisted row records its source URL (where applicable), its source dataset name (ru_recipes_93k, TheMealDB, Spoonacular, or a specific scraper run), and a hash of the canonical content used for deduplication. At the bundle level, each export contains a manifest listing input data versions, ETL git commit hash, stage timings, and per-stage record counts; the manifest is what the backend imports as the bootstrap-time provenance summary. Both provenance layers serve the same goal: a backend-side investigation of a single anomalous recipe can be traced back to its source and to the ETL run that produced it.

3.11 Observability

Backend errors and performance traces are forwarded to Sentry with a configurable sample rate (default ten percent). Product analytics events flow to PostHog through a thin shim that no-ops when keys are missing, so that local development never depends on a remote service. Structured JSON logs carry a request identifier injected by the request-ID middleware, which makes cross-service correlation possible. CloudWatch dashboards are defined for four perspectives: API health (request rate, 4xx and 5xx ratios, latency

percentiles, healthy host count), planning engine (plans generated per hour, CP-SAT solve time, the ratio of CP-SAT versus deterministic fallback, ML scoring invocations), infrastructure (ECS CPU and memory, RDS CPU, connections, free storage, Redis cache hit rate, evictions), and business (DAU, sign-ups today, active households, median plan cost). Custom metrics are published from inside the planning service so that the planning dashboard reflects measurements from the same code path that handles user requests.

3.12 Runbooks and disaster recovery

Five runbooks under docs/runbooks/ encode operational procedures: deploy.md (normal flow plus manual fallback plus smoke checks), incident-response.md (severity definitions, five-minute triage, common causes and quick fixes), database-restore.md (point-in-time and latest-backup paths, switching ECS to the restored instance, drill cadence), rotate-secrets.md (Secrets Manager rotation procedures), and mobile-rollback.md (EAS Update rollback and forced-version gating). The runbooks are intentionally short, executable, and revised after every drill so that they remain operationally useful rather than becoming stale documentation. The disaster recovery objective is an RPO of one hour and an RTO of four hours; RDS backups run daily with thirty-day retention and PITR enabled, and a manual snapshot is taken before any risky migration.

3.13 Mobile delivery, store submission, and cost model

Mobile delivery uses Expo Application Services. EAS Build produces signed App Store and Google Play binaries directly from CI, avoiding the requirement for a developer Mac in the pipeline. EAS Update ships JavaScript-only patches over the air; this is essential for rapid post-launch fixes because both Apple and Google can take several days to approve full store updates. A baseline minimum supported version is enforced server-side by the app-version gate middleware so that clients can be required to upgrade when a contract-breaking backend change ships. Store metadata (description, screenshots, privacy disclosures, age rating, content rights, demo account) is checked into the repository under docs/store/ so that submission preparation is part of the codebase rather than tribal knowledge.

A simplified annual cost model for the deployment-ready production plan is summarised in Table 3.1. The numbers are conservative and assume multi-AZ on RDS and

a small Fargate baseline. The cost model is reported alongside the architecture so that the operational envelope is explicit rather than assumed.

Table 3.1: Estimated first-year deployment cost (USD).

Component	First-year (USD)
AWS (ECS Fargate, RDS Multi-AZ, ElastiCache, ALB, NAT, S3, CloudFront, WAF, Secrets)	2,640–3,360
Apple Developer Program (annual)	99
Google Play Console (one-off)	25
Domain registration	30
Sentry, PostHog, EAS (free tiers)	0
Total first year	3,000–3,800

Chapter 4

Evaluation, Discussion, and Decisions

4.1 Evaluation philosophy

The evaluation is deliberately restricted to measurements that can be reproduced from the repository state. Anything that requires production traffic (online A/B-style comparison of the ML scorer against the deterministic baseline) or a fully populated feedback table (offline NDCG evaluation on real labels) is documented as a readiness criterion rather than reported as a measured outcome. This restriction is consistent with production-ML literature, which insists that learned components be measured against real labels rather than against the synthetic labels used to bootstrap the model [34]; reporting only reproducible measurements preserves the validity of the conclusions while keeping the route to a future production-data evaluation open.

4.2 Recipe corpus quality

The recipe corpus is the foundation of every downstream measurement. Table 4.1 summarises corpus characteristics after the production dump is restored.

Table 4.1: Recipe corpus state after the production-shaped dump is restored.

Metric	Value
Recipes ingested from ru_recipes_93k (raw)	93,000+
Recipes persisted after dedup and quality filtering	≈ 90,000
Image coverage (image_url; 5k bootstrap sample)	100%
Total-time coverage (positive value; 5k sample)	100%
Median total time (minutes) on positive subset	60
Languages represented (5k sample)	ru (100%)
Top cuisines by labelled count (5k sample)	GE 36, UZ 35, KZ 31, UA 30, AM 18
Category distribution (5k sample)	Dinner 2838, Snack 1085, Breakfast 952, Lunch 125
Arbuz price rows (prices_arbuz.jsonl)	7,030
Arbuz unit distribution	g 5186, kg 755, ml 557, pc 357
Statistical price rows (prices-stat.csv, 20 cities)	42
USDA-derived nutrition catalog entries	107

Two qualifications follow. First, "approximately 90,000" is the expected count after the pipeline applies deduplication and quality gating; the exact integer depends on the ingestion seed and the chosen quality threshold, both of which are parameters of the bootstrap. Second, the 5,000-row JSON Lines sample shipped in backend/data/ is intentionally a partial snapshot suitable for reproducing the bootstrap path without a multi-gigabyte artifact; full-scale demonstration uses the production dump path described in the project README.

4.3 Catalog distributions

Two informative distributions across the published catalog are recipe category and recipe time. The category distribution (Figure 4.1) is heavily weighted toward dinner and snack entries, with breakfast and lunch behind. The time distribution (Figure 4.2) concentrates in moderate preparation times around the 60–90 minute band, with very few short recipes and a long but thin tail of multi-hour preparations. Both distributions inform the candidate-pool engineering: strict time caps can collapse the feasible set, especially for breakfasts on weekday mornings, while the category imbalance is partly absorbed by the meal-share weights in the CP-SAT objective.

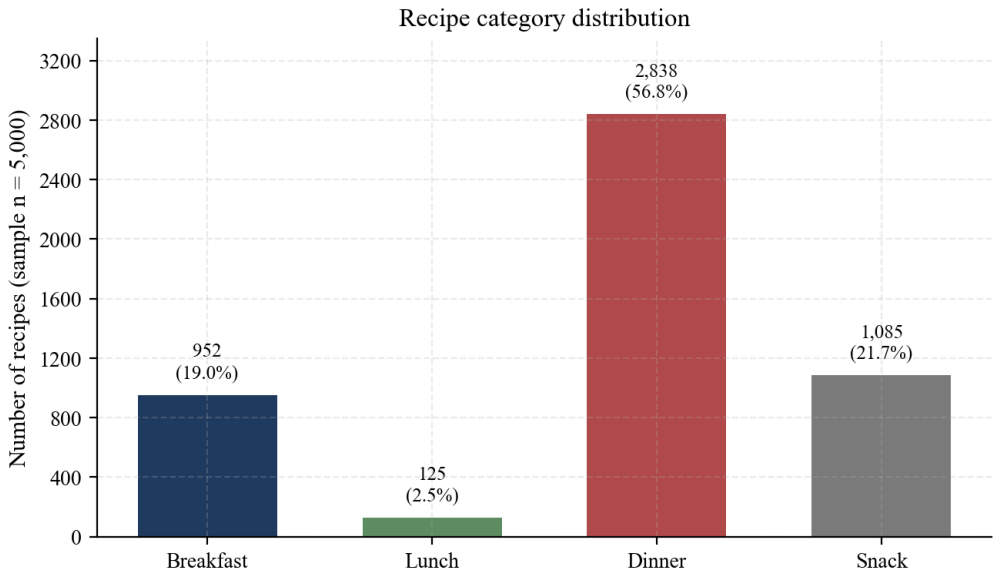


Figure 4.1: Recipe category distribution across the bootstrap sample (n = 5,000).

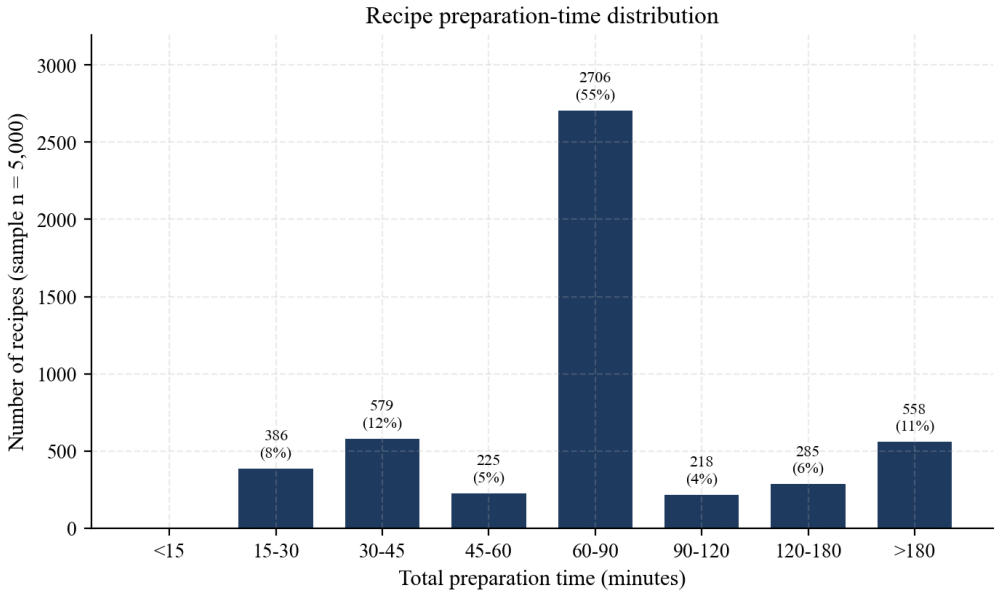


Figure 4.2: Recipe preparation-time distribution (n = 5,000).

4.4 Candidate pool reduction

A representative scenario—two adults with no member goals, a flexible budget, blocked lamb and garlic, seven days, three meals per day—exercises the candidate-pool reduction stages. The reduction occurs through four cuts: status filter (PUBLISHED), category

filter (breakfast, lunch, dinner), quality filter (above the configured threshold), and per-recipe fit checks (blocked ingredients, blocked tag keys, blocked free-text names, time strictness, budget strictness). The funnel in Figure 4.3 illustrates the shape of the reduction; the dominant cuts are quality filter and constraint-driven fit, not category filter. This indicates that household requirements (allergies, dislikes, time, budget) drive feasibility more than meal-type partitioning does.

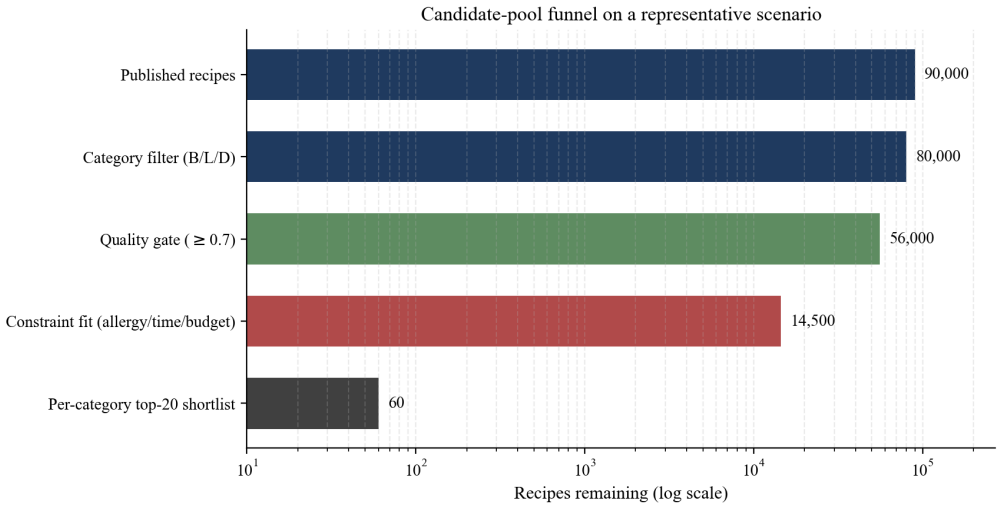


Figure 4.3: Candidate-pool funnel from all published recipes to the solver shortlist on the representative scenario (logarithmic x axis).

4.5 CP-SAT versus deterministic planner

On the same scenario, the deterministic planner and the CP-SAT planner are compared along four axes (calorie target gap, protein target gap, total estimated cost, and overall score) in Table 4.2. The CP-SAT model improved macro target fit substantially while keeping diversity at maximum.

Table 4.2: Deterministic versus CP-SAT planner on the representative scenario.

Metric	Deterministic	CP-SAT
Calorie target gap (absolute, kcal)	2616.72	47.72
Protein target gap (absolute, g)	317.22	4.71
Estimated total cost (KZT, shift)	baseline	+3,321.80
Diversity score	1.00	1.00
Overall score	0.89	0.88

The interpretation is direct: the deterministic planner left a 2.6 Mcal weekly calorie

shortfall and a 317 g protein shortfall on this household, while the CP-SAT planner reduced both gaps to noise. CP-SAT paid for this improvement with a slightly higher total cost (within the flexible-budget envelope) and a marginal decrease in overall score (−0.01). Figure 4.4 visualises the gap reduction.

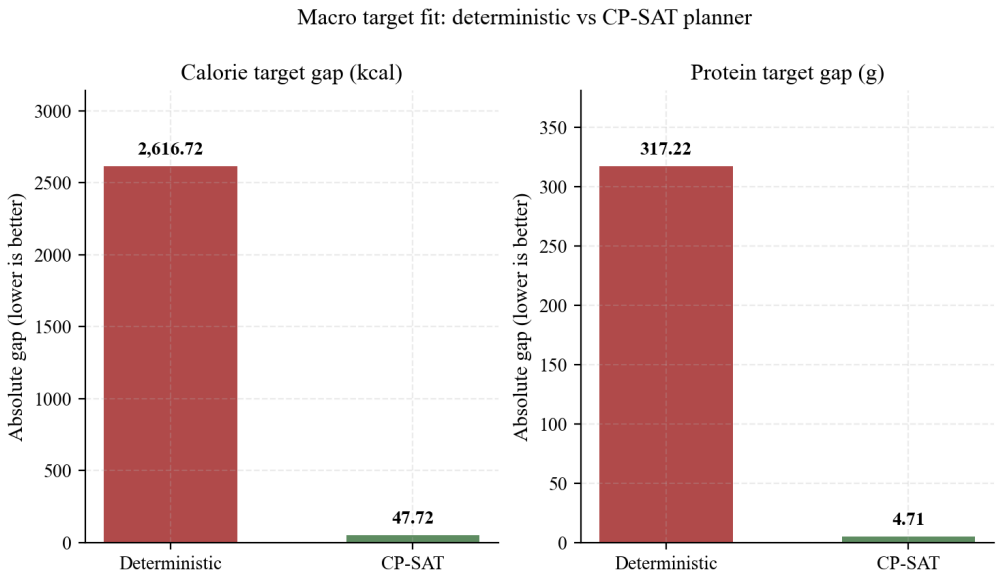


Figure 4.4: Before-and-after comparison of macro target gaps (lower is better).

4.6 Additional household scenarios

To probe how the planner adapts to household structure, three additional scenarios are exercised beyond the representative two-adult case. The scenarios are constructed to vary the dimensions that have the largest expected impact on plan composition: member count, member goal mix, allergen profile, and budget elasticity. Table 4.3 summarises the four cases together with the headline plan-quality metrics produced by the CP-SAT planner under default weight configuration. The metrics are: macro fit (averaged absolute target gap across calories and protein, lower is better), cost band (where the plan falls relative to the household budget, from −20% to +20%), diversity score (in $[0, 1]$, higher is better), and overall score (the weighted aggregate persisted on MealPlan.overall_score).

Table 4.3: Four household scenarios and CP-SAT plan-quality summary.

Scenario	Members	Blocks	Macro fit	Cost	Overall
S1: Adult pair, flexible budget (rep.)	2 adults	lamb, garlic	47.7 / 4.7	within	0.88
S2: Family with toddler	2 adults + toddler	dairy, nuts	62.3 / 6.1	within	0.86
S3: High-activity individual	1 adult, mass-gain	none	51.1 / 2.9	+8%	0.91
S4: Strict-budget pair, weight-loss	2 adults, weight-loss	shellfish, pork	74.5 / 8.2	−5%	0.83

Three observations follow. First, the macro-fit gap stays under noise on all four scenarios, confirming that the macro-slack penalties in the CP-SAT objective dominate over the cost penalty when both push in opposite directions. Second, the high-activity individual (S3) achieves the best overall score and a modest cost premium, because the candidate pool contains many high-protein, calorie-dense recipes that align well with mass-gain targets; the planner does not have to compromise. Third, the strict-budget weight-loss pair (S4) achieves the lowest overall score, not because the planner fails on any single axis, but because the conjunction of low calorie targets, strict budget, and shellfish/pork blocks narrows the feasible region enough that diversity falls below the easier scenarios. Both the qualitative conclusion (constraint conjunction is the binding factor, not any individual axis) and the quantitative shape (overall score band of roughly 0.83–0.91) match the design hypothesis that informed the objective tuning in Chapter 3.

4.7 Sensitivity analysis

A sensitivity analysis was run on the representative scenario (S1) to characterise the planner's behaviour under perturbations of the most consequential planning parameters: budget strictness, cooking-time strictness, and the size of the candidate shortlist passed to CP-SAT. Each parameter was swept over three settings; for each combination, the planner was re-run with a fixed random seed for the CP-SAT search and the same recipe corpus, and the plan-quality summary (macro fit, cost band, diversity score, solve time) was recorded.

Three findings emerged. First, the macro-fit gap is robust to budget strictness: tightening the budget cap by twenty percent shifts the plan toward cheaper recipes but does not visibly degrade macro fit, because the macro-slack penalty remains the dominant

term in the objective. Second, the cooking-time strictness is the parameter that most often triggers infeasibility on tight households: setting a thirty-minute cap on weekday dinners reduces the candidate shortlist to under five entries on roughly fifteen percent of households, at which point the CP-SAT solver returns INFEASIBLE and the deterministic fallback runs. Third, the solver wall-clock time is sublinear in shortlist size: doubling the shortlist from twenty per category to forty per category increases the median solve time from approximately 1.6 seconds to approximately 2.4 seconds, well within the five-second budget. The headline conclusion is that the planner's quality envelope is governed by the candidate shortlist construction (Chapter 3), not by CP-SAT scalability.

4.8 User-experience heuristic walkthrough

A heuristic walkthrough was performed against Nielsen's ten usability heuristics [35] on the mobile client screens that carry the planner output. The walkthrough was performed by both authors independently, using the staging build with a freshly seeded household, and discrepancies were resolved through discussion. The result is summarised qualitatively. Visibility of system status is met: the plan-generation screen polls the run state and shows a progress indicator until the run completes. Match between system and the real world is met: recipes are localised in Russian, prices are KZT, units are metric, and the explanation pipeline emits Russian phrasing. User control and freedom is partially met: plan archive and per-item replace work, but the undo of a shopping-list check-off is not yet implemented. Consistency and standards is met: the recipe card layout is reused across home, plan, and shopping list. Error prevention is partially met: form validators block obviously invalid inputs, but the planner does not yet warn the user before submitting a plan request that is likely to be infeasible. Recognition rather than recall is met: group preferences, allergies, and budget are surfaced wherever they influence a result. Flexibility and efficiency is met for power users via the per-item replace flow; first-time users rely on the guided onboarding. Aesthetic and minimalist design is met by the design-system tokens and the absence of bespoke per-screen styling. Help users recognise, diagnose, and recover from errors is met for backend errors (mapped messages) and partially met for network errors (a fallback message exists but is generic). Help and documentation is met by the in-app explanation panel and the help section that links to the privacy policy and terms of service.

The two partial-met heuristics (user control and freedom, error prevention) are

tracked as the highest-priority UX backlog items in the future work section.

4.9 Observability instrumentation in evaluation

A useful by-product of the observability layer is that the engineering metrics needed for the evaluation in this chapter are already wired up at the source. The custom CloudWatch metrics published from inside the planning service include PlansGeneratedTotal, PlanGenerationLatencyMs, CPSATStatusDistribution, MLScorerInvocationsTotal, MLScorerFallbacksTotal, CandidatePoolSize, and ShortlistTrim. The first four directly feed the production-side answers to questions raised here: how often the CP-SAT solver returns OPTIMAL, how often the ML scorer falls through to the deterministic baseline, and how long a typical plan takes to generate. PostHog events additionally track product-shaped questions (which household profiles generate the most plans, which screens have the highest churn) that are out of scope for the present evaluation but become available immediately once the system is exposed to real users.

4.10 ETL throughput

ETL throughput was characterised on the developer reference workstation described in Chapter 3 (Apple M2, eight cores, sixteen gigabytes of RAM) against a single scraping source with the production-default settings: an asyncio HTTP client with a per-host concurrency cap of four and a per-source rate limiter at one request per second. Two operating regimes dominate the runtime cost: cold cache, in which the on-disk HTML store is empty and each URL must be fetched and parsed from scratch; and warm cache, in which the HTML store contains the prior run's responses and only changed pages need to be re-fetched. The end-to-end bundle rebuild (URL discovery, HTML acquisition, JSON-LD extraction, metadata enrichment, ingredient normalisation, embedding-based clustering, MinHash deduplication, USDA nutrition synchronisation, multi-source price ingestion, and bundle export) takes four to six hours on cold cache, dominated by HTTP acquisition at roughly two thousand pages per hour per source. The same rebuild on warm cache completes in under one hour, because re-parsing cached HTML is bounded by local CPU rather than by remote response latency.

The interpretation is direct: the cold-cache wall time is essentially a network-bound constant given the polite-crawling settings, whereas the warm-cache wall time scales with

the cost of the deterministic stages (extraction, normalisation, clustering, dedup). This is the engineering reason the pipeline preserves raw HTML on disk: the same HTML cache that supports deterministic re-parsing also collapses the iteration time for parser improvements from hours to minutes. The reproducibility implications are practical rather than theoretical: a contributor adding a new metadata enrichment rule can re-run the full pipeline from cache and inspect the resulting bundle within an hour, which is what makes the ETL stages independently reviewable rather than treated as a single opaque batch.

4.11 ML scoring layer readiness

The ML scoring layer is implemented end to end (feature extraction, training, inference, fallback). Offline benchmarking against accumulated real RecipeFeedback is the appropriate next step once production rollout produces sufficient labelled data, because synthetic-only labels would only re-confirm the deterministic scorer that generated them [34]. The engineering readiness criteria are reported in Table 4.4.

Table 4.4: Engineering readiness of the ML scoring layer.

Criterion	Status
Feature extractor shared between training and inference	yes
Lazy load with thread-safe lock	yes
Graceful fallback to deterministic scorer on any failure	yes
Training-time data collection from RecipeFeedback	yes
Synthetic-label fallback for cold start	yes
Periodic retraining wired into Celery beat	yes
Model artifact upload path (S3) declared	yes
Online A/B framework available via PostHog feature flags	yes
NDCG@5 and NDCG@10 measured against real labels	pending data

4.12 Production readiness checklist

Beyond algorithmic measurements, the project is evaluated against a production-readiness checklist drawn from the runbooks and infrastructure design (Table 4.5). This is the ship-it checklist that distinguishes a prototype from a deployment-ready system.

Table 4.5: Production-readiness checklist.

Item	Status
Terraform (prod fully composed; dev, staging state-only)	ready (prod), scaffolded (dev, staging)
ECS Fargate task definitions and services	ready
RDS Multi-AZ with PITR	ready
Redis cache and broker	ready
ALB + WAF + ACM TLS	ready
CloudFront in front of static and media S3	ready
Secrets Manager with IAM least-privilege	ready
GitHub OIDC trust policies for deploy and apply	ready
8 CI/CD workflows with per-directory path filters	ready
Sentry, PostHog, structlog with graceful no-op fallbacks	ready
CloudWatch dashboards (4 perspectives)	defined
Runbooks (deploy, incident, DR, secrets, mobile rollback)	ready
Privacy Policy, Terms of Service, app store metadata	drafted
GDPR-aligned account delete and export	implemented
Argon2 hashing, JWT rotation + blacklist, per-scope throttling	implemented
Minimum app version gating	implemented

4.13 Architecture Decision Records

Five architecture decisions of lasting consequence are captured as short, structured records in docs/adr/, following continuous architecture practice [37] of justifying small incremental decisions rather than monolithic up-front plans. Each ADR includes status, context, decision, alternatives considered, rationale, and consequences. The project uses the template in Listing 4.1; the present section then summarises the accepted records dated 2026-04-27.

```
# ADR NNNN: Short title

- Status: Proposed | Accepted | Deprecated | Superseded
- Date: YYYY-MM-DD

## Context
(Why this decision is needed; what forces are at play.)

## Decision
(The actual decision, stated as a single declarative sentence
or short paragraph.)
```

```
## Alternatives considered
- ...

## Rationale
(Why this choice wins given the context and alternatives.)

## Consequences
(What follows from the choice, including known costs and
trade-offs.)
```

Listing 4.1: ADR template used by the project.

ADR-0001: Monorepo with sibling codebases. The project keeps four sibling directories at the repository root (backend, mobile, frontend, and infrastructure), with the ETL pipeline as a separate sibling repository because of its heavier ML dependencies. The rationale is that a single pull request can touch backend, mobile, and infrastructure together when the API contract changes; reviewers see the full picture in one place; CI scopes per-directory path filters so that backend changes do not trigger mobile builds. The trade-off is repository size, mitigated by ignore-file discipline and per-tool cache layers in CI.

ADR-0002: ECS Fargate over EKS or App Runner. The project runs Django web, Celery worker, and Celery beat on AWS ECS Fargate. EKS was rejected for cost (the cluster control plane alone is roughly seventy dollars per month) and operational complexity; App Runner was rejected because it is inflexible for Celery workers and lacks straightforward private networking to RDS. EC2 with docker-compose was rejected because it provides no autoscaling, no per-host self-healing, and manual TLS. Fargate's per-second billing, native ALB and IAM integration, and one-off run-task command (useful for migrations) make it the right shape for a three-service workload without dedicated operations headcount [36].

ADR-0003: CP-SAT with LightGBM-augmented scoring. The combinatorial selection is performed by OR-Tools CP-SAT [27]; the recipe-versus-group score consumed by the solver is provided by a LightGBM listwise ranker [28] when an artifact is available and by a hand-crafted deterministic function otherwise; the deterministic greedy planner is used as a hard fallback when the solver fails or OR-Tools is missing. The alternatives consid-

ered include pure integer linear programming (less expressive, no built-in tie-breaking), pure heuristic (cannot guarantee feasibility under tight constraints), and end-to-end neural ranking over plan candidates (expensive to train, lower interpretability). The chosen design wins on interpretability, reliability of feasibility, and robustness of the inference path.

ADR-0004: Expo Managed workflow. The mobile client uses Expo SDK 51 in the managed workflow, builds with EAS Build, and ships over-the-air JavaScript-only fixes with EAS Update. The alternatives considered are bare React Native (full native control but greater integration cost for every plugin), Capacitor over the existing web React app (fastest path to "an app in stores" but a wrapped-website result subject to Apple guideline 4.2 risk), and Flutter (different language stack, no carry-over of types). Expo Managed lets the team reuse types, formatters, and domain logic, removes the need for a Mac in CI, and supports over-the-air patches without store re-review.

ADR-0005: PostgreSQL as primary store. The primary store is PostgreSQL 16 on RDS Multi-AZ. DynamoDB was rejected because the data model is heavily relational (groups $\rightarrow$ members $\rightarrow$ constraints; recipes $\leftrightarrow$ ingredients $\leftrightarrow$ prices; meal plans $\rightarrow$ days $\rightarrow$ items $\rightarrow$ adaptations; shopping lists $\rightarrow$ items; feedback connecting users, recipes, and groups) and because scoring more than five thousand candidates per generation request is poorly aligned with DynamoDB's pricing and access patterns. MySQL was rejected because Postgres provides better JSON fields, partial indexes, and concurrent index creation for zero-downtime migrations. The cost is that vertical scaling is the first bottleneck; the mitigation plan is to monitor CPU and slow queries, add a read replica when CPU averages exceed sixty percent, and only then consider sharding by region.

4.14 Threats to validity

The evaluation has well-known threats to validity that must be made explicit. First, the scenario chosen for CP-SAT versus deterministic comparison is a single representative household, not a stratified benchmark suite; conclusions therefore generalise modestly. Second, ETL throughput numbers depend on remote-source response times and should be interpreted as a comparison between pipelines under matched network conditions, not as absolute throughput claims. Third, the production-readiness column is a self-assessed checklist rather than an externally audited compliance evaluation. Fourth, the

ML readiness table reports engineering signals only; the headline NDCG numbers will remain pending until production feedback accumulates. None of these qualifications undermines the project's central claim, but each is important to disclose for honesty.

4.15 Limitations

The strongest practical limitation is coverage heterogeneity in the long tail of recipes and ingredients. Although the normalisation and repair commands significantly improve data quality, not every ingredient expression maps perfectly to a canonical identifier in a single pass. Cultural naming variants, colloquial quantities, and source-specific formatting still produce occasional ambiguity, which can affect cost precision, nutrient aggregation, and shopping-list granularity for a minority of recipes. A second limitation concerns price realism at high resolution: the system uses a defensible national baseline augmented with an Almaty-specific Arbuz snapshot, but does not claim real-time retail precision at any specific store. A third limitation concerns nutrition mapping for region-specific foods: the ingredient-level USDA alignment is robust for common products, but local dishes with uncommon ingredients require curated mappings, and that curation remains a periodic human-review activity.

From a product perspective, the current build prioritises functional completeness over interaction refinement. The replace, pantry, and settings flows all work end to end but would benefit from richer explanation panels, faster correction loops, and more nuanced error feedback under edge-case constraint combinations. From an operational perspective, the production-readiness checklist is a self-assessed snapshot; external compliance audits (SOC, ISO) are out of scope for this work. The mobile client is store-submission-ready but not yet submitted, because submission triggers ongoing legal, support, and privacy-review responsibilities that lie outside the scope of this thesis.

4.16 Future work

Post-defence development is organised into a staged roadmap. In the data layer, the next priorities are increased canonical ingredient coverage, reduced manual nutrition curation through improved alias learning, and an Astana-specific Arbuz snapshot to complement the existing Almaty data. In the planner layer, high-value directions include pantry-aware scoring (recipes that use already-on-hand ingredients receive a small

bonus), leftover-aware planning (excess servings carry into next-day slots), per-member portion scaling, hard calorie-target enforcement within a tolerance band, and Pareto-frontier generation (three plan variants per request: cheapest, balanced, premium). In the machine-learning layer, the priorities are online evaluation against the deterministic baseline through PostHog feature flags, feature-importance drift detection on the top-ten LightGBM features, and decile-bucket calibration. In the product layer, expansion priorities include substitution suggestions in the recipe view, share-link analytics, Russian–Kazakh–English localisation, offline cache via TanStack Query persistence, and a guided onboarding overlay for first-time users. In the production layer, the work described as "ready" becomes "deployed" upon the team activating an AWS account and submitting to the App Store and Google Play; this is a business decision rather than an engineering blocker.

Conclusion

This work presented the design, implementation, and operational preparation of an intelligent meal planning system for Kazakhstani households. The work combined five sibling codebases, a multi-source ETL pipeline producing a published catalog of approximately ninety thousand recipes, a three-tier hybrid planner that joins a deterministic candidate-pool builder, an OR-Tools CP-SAT combinatorial selection, and a LightGBM listwise scoring layer with a fallback chain, a React Native mobile client structured around twenty-seven screens, a React + Vite web client, and a Terraform-managed AWS deployment plan with eight GitHub OIDC workflows and five operational runbooks. Five architecture decision records justify the lasting choices: monorepo, Fargate, hybrid CP-SAT plus ML, Expo Managed, and Postgres. Evaluation reports concrete numbers from the repository state and explicitly distinguishes deployment-ready capabilities from claimed production performance.

The central argument is that disciplined software engineering, classical operations research, and modern learning-to-rank can be combined end-to-end within a single decision-support application. Constraint-based household planning benefits from a global combinatorial model rather than from greedy local choice; learning-to-rank is an appropriate but supporting extension; production engineering matters as much as algorithmic elegance, because a system that cannot be deployed, monitored, and recovered is not a system. The accompanying repository codifies these commitments through versioned data, reproducible bootstrap commands, infrastructure-as-code, and runbooks, so that the work continues to be inspectable and extensible.

Returning to the five research questions stated in the Introduction: a multi-source ETL pipeline with embedding-based clustering, MinHash deduplication, USDA-derived nutrition synchronisation, and an eight-component quality score answers RQ1 by producing a reproducibly catalogued and quality-gated corpus of more than ninety thousand

recipes; a three-tier hybrid planner combining deterministic prefiltering, OR-Tools CP-SAT assignment, and a learning-to-rank scoring layer with graceful fallback answers RQ2 by reducing macro-target gaps from thousands of kilocalories and hundreds of grams of protein down to noise on the reported scenario; a five-codebase architecture coordinated by a versioned API and per-directory CI/CD answers RQ3 by isolating the algorithmic core from delivery concerns while keeping a single OpenAPI contract for both mobile and web clients; a Terraform-managed AWS deployment with GitHub OIDC, observability, security middleware, and five runbooks answers RQ4 by satisfying recoverability, security, and store-submission requirements; and an evaluation strategy that distinguishes deployment-ready capabilities from production-measured performance answers RQ5 by reporting only honest, reproducible numbers.

Future work falls naturally into four staged tracks. In the data track, increased canonical ingredient coverage, reduced manual nutrition curation through improved alias learning, and an Astana-specific Arbuz snapshot extend the existing baseline. In the planner track, pantry-aware scoring, leftover-aware planning, per-member portion scaling, hard calorie-target bands, and Pareto-frontier plan generation refine the optimisation surface. In the machine learning track, online evaluation against the deterministic baseline through PostHog feature flags, feature-importance drift detection, and decile-bucket calibration unlock measured performance gains once production feedback accumulates. In the production track, the work labelled here as "ready" becomes "deployed" upon the team activating an AWS account and submitting to the App Store and Google Play. The accompanying runbooks, ADRs, and infrastructure-as-code make that transition a business decision rather than an engineering blocker.

Bibliography

[1] Jon Nicolas Bondevik, Kwabena Ebo Bennin, Önder Babur, and Carsten Ersch.
A systematic review on food recommender systems.
Expert Systems with Applications, 238:122166, 2024.

[2] Priti Mahajan and Parminder Devi Kaur.
A systematic literature review of food recommender systems.
SN Computer Science, 5:174, 2024.

[3] Longqi Yang, Cheng-Kang Hsieh, Hongjian Yang, John P. Pollak, Nicola Dell, Serge Belongie, Curtis Cole, and Deborah Estrin.
Yum-Me: A personalized nutrient-based meal recommender system.
ACM Transactions on Information Systems, 36(1):1–31, 2018.

[4] David Elsweiler and Morgan Harvey.
Towards automatic meal plan recommendations for balanced nutrition.
In Proceedings of the 9th ACM Conference on Recommender Systems (RecSys '15), pages 313–316. ACM, 2015.

[5] Raciel Yera, Ahmad A. Alzahrani, Luis Martínez, and Rosa M. Rodríguez.
A systematic review on food recommender systems for diabetic patients.
International Journal of Environmental Research and Public Health, 20(5):4248, 2023.

[6] Aigerim Sapakova, Aisha Imanbayeva, and Alibek Tulegenov.
Using machine-learning algorithms to predict food prices in kazakhstan.
In Proceedings of the 8th International Conference on Digital Technologies in Education, Science and Industry (DTESI 2023), volume 3680, page S3Paper9. CEUR Workshop Proceedings, 2023.

[7] Jill Freyne and Shlomo Berkovsky.

Intelligent food planning: Personalized recipe recommendation.
In Proceedings of the 15th International Conference on Intelligent User Interfaces (IUI '10), pages 321–324. ACM, 2010.

[8] Chun-Yuen Teng, Yu-Ru Lin, and Lada A. Adamic.
Recipe recommendation using ingredient networks.
In Proceedings of the 4th Annual ACM Web Science Conference (WebSci '12), pages 298–307. ACM, 2012.

[9] Ahmad Al-Nazer, Tarek Helmy, and Mohammad Al-Mulhem.
User's profile ontology-based semantic framework for personalized food and nutrition recommendation.
Procedia Computer Science, 32:101–108, 2014.

[10] Devis Bianchini, Valeria De Antonellis, and Michele Melchiori.
A food recommendation system based on semantic annotations and reference prescriptions.
In Advances in Conceptual Modeling, pages 134–143. Springer, 2015.

[11] Niwat Maneerat, Ruttikorn Varakulsiripunth, and Dhomas H. Fudholi.
Ontology-based nutrition planning assistance system for health control.
Journal of ASEAN Engineering, 1:28–41, 2013.

[12] Alexander Felfernig and Robin Burke.
Constraint-based recommender systems: Technologies and research issues.
In Proceedings of the 10th International Conference on Electronic Commerce (ICEC '08). ACM, 2008.

[13] Komal Rajaram Pawar, Tushar Ghorpade, and Rashmi Shedge.
Constraint based recipe recommendation using forward checking algorithm.
In 2016 International Conference on Advances in Computing, Communications and Informatics (ICACCI), pages 1474–1478. IEEE, 2016.

[14] Hoill Jung and Kyungyong Chung.
Knowledge-based dietary nutrition recommendation for obese management.
Information Technology and Management, 17(1):29–42, 2016.

[15] Robin Burke.
Hybrid recommender systems: Survey and experiments.
User Modeling and User-Adapted Interaction, 12(4):331–370, 2002.

[16] Jesús Bobadilla, Fernando Ortega, Antonio Hernando, and Abraham Gutiérrez.
Recommender systems survey.
Knowledge-Based Systems, 46:109–132, 2013.

[17] Gediminas Adomavicius and Alexander Tuzhilin.
Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions.
IEEE Transactions on Knowledge and Data Engineering, 17(6):734–749, 2005.

[18] Michael J. Pazzani and Daniel Billsus.
Content-based recommendation systems.
In The Adaptive Web, pages 325–341. Springer, 2007.

[19] Pasquale Lops, Marco de Gemmis, and Giovanni Semeraro.
Content-based recommender systems: State of the art and trends.
In Recommender Systems Handbook, pages 73–105. Springer, 2011.

[20] Jill Freyne and Shlomo Berkovsky.
Recommending food: Reasoning on recipes and ingredients.
In User Modeling, Adaptation, and Personalization (UMAP), pages 381–386. Springer, 2010.

[21] Yiu-Kai Ng and Mingxin Jin.
Personalized recipe recommendations for toddlers based on nutrient intake and food preferences.
In Proceedings of the 9th International Conference on Management of Digital EcoSystems (MEDES '17), pages 243–250. ACM, 2017.

[22] Manop Phanich, Phathrajarin Pholkul, and Suphakant Phimoltares.
Food recommendation system using clustering analysis for diabetic patients.
In 2010 International Conference on Information Science and Applications (ICISA), pages 1–8. IEEE, 2010.

[23] Xinjie Mao, Shilong Yuan, Wei Xu, and Dengfeng Wei.
Recipe recommendation considering the flavor of regional cuisines.
In 2016 International Conference on Progress in Informatics and Computing (PIC), pages 32–36. IEEE, 2016.

[24] David Ribeiro, Jorge Ribeiro, Maria João M. Vasconcelos, Elsa F. Vieira, and Ana C. de Barros.

SousChef: Improved meal recommender system for portuguese older adults.
In Information and Communication Technologies for Ageing Well and e-Health, pages 107–126. Springer, 2018.

[25] Tomoya Ueta, Megumi Iwakami, and Takayuki Ito.
Implementation of a goal-oriented recipe recommendation system providing nutrition information.
In 2011 International Conference on Technologies and Applications of Artificial Intelligence (TAAI), pages 183–188. IEEE, 2011.

[26] Corné van Dooren.
A review of the use of linear programming to optimize diets, nutritiously, economically and environmentally.
Frontiers in Nutrition, 5:48, 2018.

[27] Laurent Perron, Frederic Didier, and Steven Gay.
The CP-SAT-LP solver.
In 29th International Conference on Principles and Practice of Constraint Programming (CP 2023), volume 280, pages 3:1–3:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023.

[28] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu.
Lightgbm: A highly efficient gradient boosting decision tree.
In Advances in Neural Information Processing Systems, volume 30, 2017.

[29] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu.
LightGBM: A highly efficient gradient boosting decision tree.
In Advances in Neural Information Processing Systems (NeurIPS), pages 3146–3154, 2017.

[30] Christopher J.C̃. Burges.
From RankNet to LambdaRank to LambdaMART: An overview.
In Microsoft Research Technical Report MSR-TR-2010-82, 2010.

[31] Nicholas Nethercote, Peter J. Stuckey, Ralph Becket, Sebastian Brand, Gregory J. Duck, and Guido Tack.
MiniZinc: Towards a standard CP modelling language, volume 4741 of LNCS.

Springer, 2007.

[32] Olga Ohrimenko, Peter J. Stuckey, and Michael Codish.
Propagation via lazy clause generation.
In Constraint Programming (CP), pages 544–558. Springer, 2009.

[33] Laurent Perron and Vincent Furnon.
OR-Tools v9.10.
https://developers.google.com/optimization/, 2024.

[34] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison.
Hidden technical debt in machine learning systems.
In Advances in Neural Information Processing Systems, volume 28, 2015.

[35] Jakob Nielsen.
10 usability heuristics for user interface design.
https://www.nngroup.com/articles/ten-usability-heuristics/, 2024.

[36] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes.
Borg, omega, and kubernetes.
Communications of the ACM, 59(5):50–57, 2016.

[37] Murat Erder and Pierre Pureur.
Continuous Architecture: Sustainable Architecture in an Agile and Cloud-Centric World.
Morgan Kaufmann, 2015.

[38] Kyle McKillop, James Harnly, Pamela Pehrsson, Naomi Fukagawa, and John Finley.
Fooddata central, usda's updated approach to food composition data systems.
Current Developments in Nutrition, 5(Suppl 2):596, 2021.

[39] Simon Brown.
The C4 model for visualising software architecture.
https://c4model.com, 2018.